

Lecture: 12

Deadlock Detection in Distributed Systems



Rajiv Misra

Dept. of Computer Science &
Engineering

Indian Institute of Technology Patna

rajivm@iitp.ac.in

Preface

Recap of Previous Lecture:

- In the previous lecture, we have discussed about '**Checkpointing and Rollback Recovery**'. Also discuss different **Rollback Recovery Schemes** i.e. (i) Checkpoint based and (ii) Log based

Content of this Lecture:

- In this lecture, we will discuss about the basic fundamentals of '**Deadlock Detection in Distributed Systems**'.
- Also discuss different classes of algorithms such as path-pushing, edge-chasing, diffusion computation, and global state detection.

Introduction

- **Deadlocks** are a fundamental problem in distributed systems.
- A process may request resources in any order, which may not be known a priori and a process can request resource while holding others.
- If the sequence of the allocations of resources to the processes is not controlled, deadlocks can occur.
- A deadlock is a state where a set of processes request resources that are held by other processes in the set.

System Model

- A distributed program is composed of a **set of n asynchronous processes $p_1, p_2, \dots, p_i, \dots, p_n$** that communicates by message passing over the communication network.
- **Without loss of generality** we assume that each process is running on a different processor.
- The processors do not share a common global memory and communicate solely by passing messages over the communication network.

Contd...

- There is **no physical global clock** in the system to which processes have instantaneous access.
- The communication medium **may deliver messages out of order**, messages may be **lost garbled or duplicated** due to timeout and retransmission, processors may fail and communication links may go down.
- Following assumptions are made:
 1. The systems have only reusable resources.
 2. Processes are allowed to make only exclusive access to resources.
 3. There is only one copy of each resource.
 4. A process can be in two states: **running** or **blocked**.
 5. In the running state (also called **active** state), a process has all the needed resources and is either executing or is ready for execution.
 6. In the blocked state, a process is waiting to acquire some resource.

Wait-For-Graph (WFG)

- The state of the system can be **modeled by directed graph, called a wait for graph (WFG).**
- In a WFG , nodes are processes and there is a directed edge from node P_1 to node P_2 if P_1 is blocked and is waiting for P_2 to release some resource.
- **A system is deadlocked** if and only if there exists a **directed cycle or knot** in the WFG.

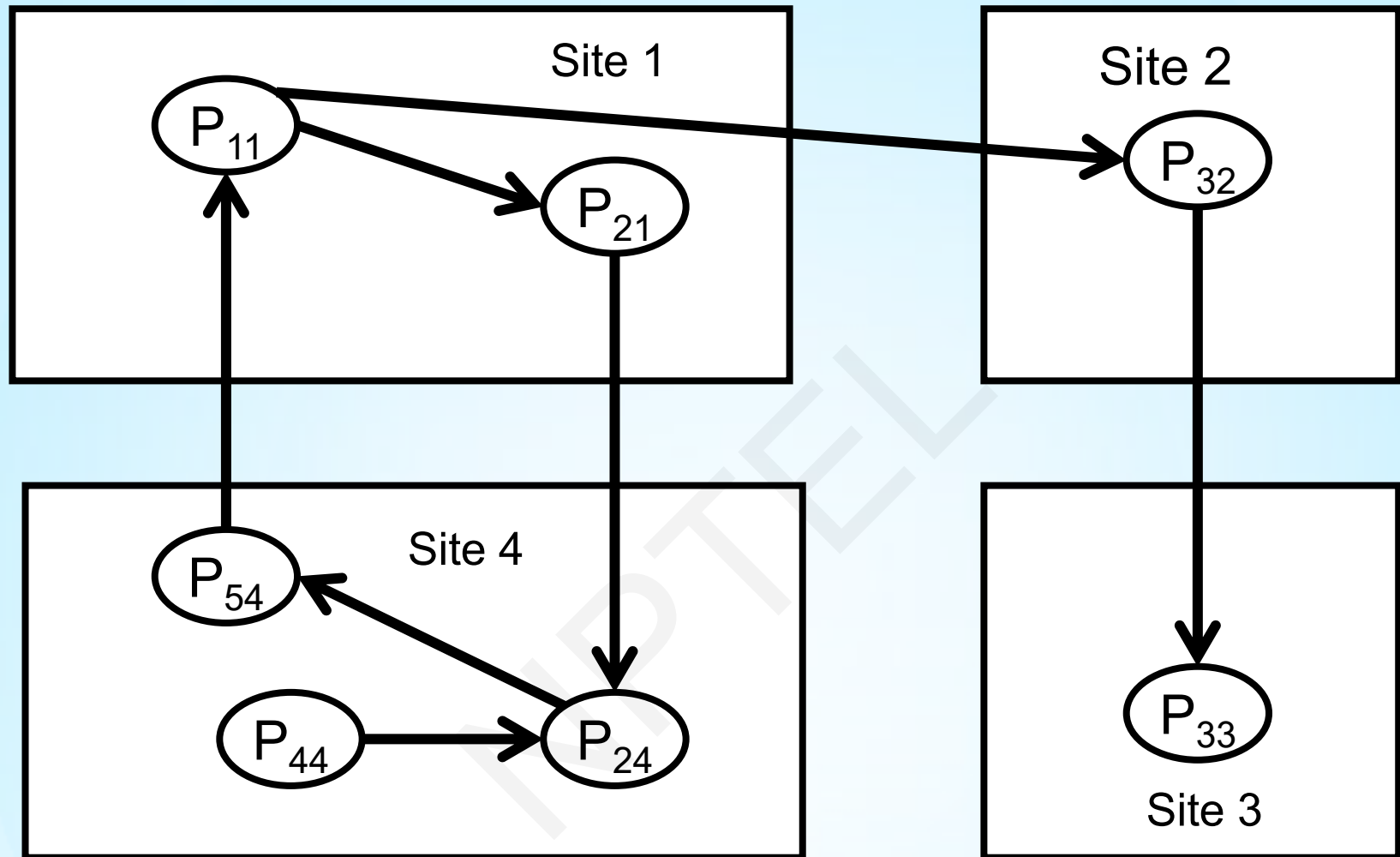


Figure 12.1 shows a WFG, where process P_{11} of site 1 has an edge to process P_{21} of site 1 and P_{32} of site 2 is waiting for a resource which is currently held by process P_{21} . At the same time process P_{32} is waiting on process P_{33} to release a resource. If P_{21} is waiting on process P_{11} , then processes P_{11} , P_{32} and P_{21} form a cycle and all the four processes are involved in a deadlock depending upon the request model.

Preliminaries

Deadlock Handling Strategies

- There are three strategies for handling deadlocks, viz., **(i) deadlock prevention, (ii) deadlock avoidance, and (iii) deadlock detection.**
- **Handling of deadlock** becomes highly complicated in distributed systems because no site has accurate knowledge of the current state of the system and because every inter-site communication involves a finite and unpredictable delay.
- **Deadlock prevention** is commonly achieved either by having a process acquire all the needed resources simultaneously before it begins executing or by preempting a process which holds the needed resource.
- This approach is highly inefficient and impractical in distributed systems.

Contd...

- In **deadlock avoidance** approach to distributed systems, a resource is granted to a process if the resulting global system state is safe (note that a global state includes all the processes and resources of the distributed system).
- However, due to several problems, **deadlock avoidance is impractical** in distributed systems.
- **Deadlock detection** requires examination of the status of process-resource interactions for presence of cyclic wait.
- **Deadlock detection** in distributed systems seems to be the best approach to handle deadlocks in distributed systems.

Issues in Deadlock Detection

- Deadlock handling using the approach of deadlock detection entails addressing two basic issues: First, detection of existing deadlocks and second resolution of detected deadlocks.
- Detection of deadlocks involves addressing two issues: Maintenance of the WFG and searching of the WFG for the presence of cycles (**or knots**).

Correctness Criteria

- A deadlock detection algorithm must satisfy the following two conditions:

(i) **Progress (No undetected deadlocks):**

- The algorithm must detect all existing deadlocks in finite time.
- In other words, after all wait-for dependencies for a deadlock have formed, the algorithm should not wait for any more events to occur to detect the deadlock.

(ii) **Safety (No false deadlocks):**

- The algorithm should not report deadlocks which do not exist (called *phantom* or *false* deadlocks).

Resolution of a Detected Deadlock

- Deadlock resolution involves **breaking existing wait-for dependencies** between the processes to resolve the deadlock.
- It involves **rolling back one or more deadlocked processes** and **assigning their resources to blocked processes** so that they can resume execution.

Models of Deadlocks

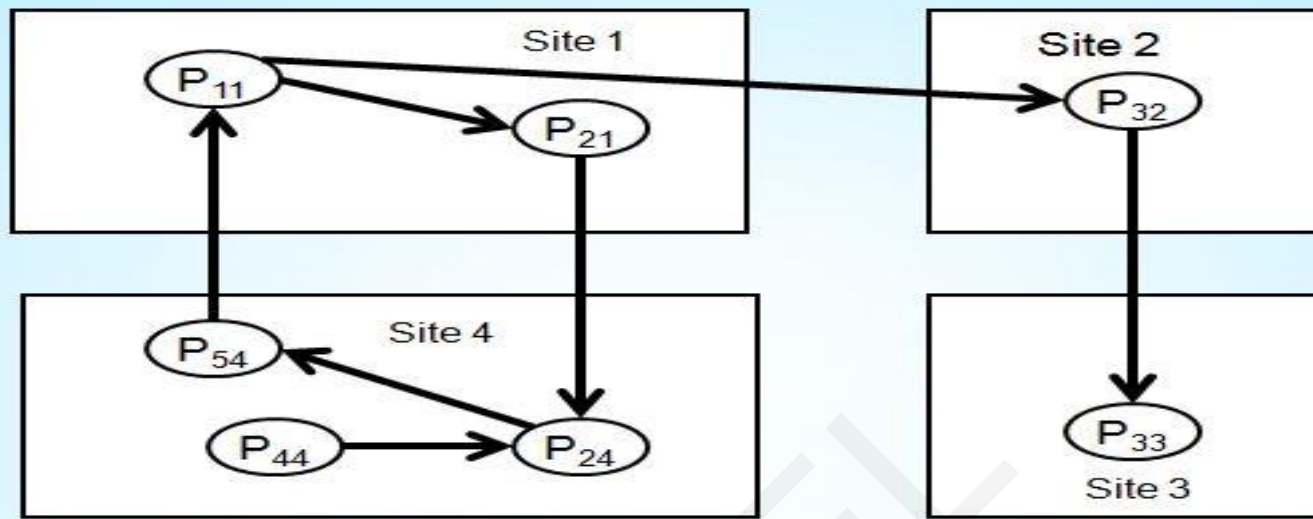
- Distributed systems allow several kinds of resource requests. A process might require a single resource or a combination of resources for its execution. A hierarchy of request models are described as follows:

(i) The Single Resource Model

- In the single resource model, a process can have at most one outstanding request for only one unit of a resource.
- Since the maximum out-degree of a node in a WFG for the single resource model can be 1, the presence of a cycle in the WFG shall indicate that there is a deadlock.

(ii) The AND Model

- In the AND model, a process can request for more than one resource simultaneously and the request is satisfied only after all the requested resources are granted to the process.
- The out degree of a node in the WFG for AND model can be more than 1.
- The presence of a cycle in the WFG indicates a deadlock in the AND model.
- Since in the single-resource model, a process can have at most one outstanding request, the AND model is more general than the single-resource model.



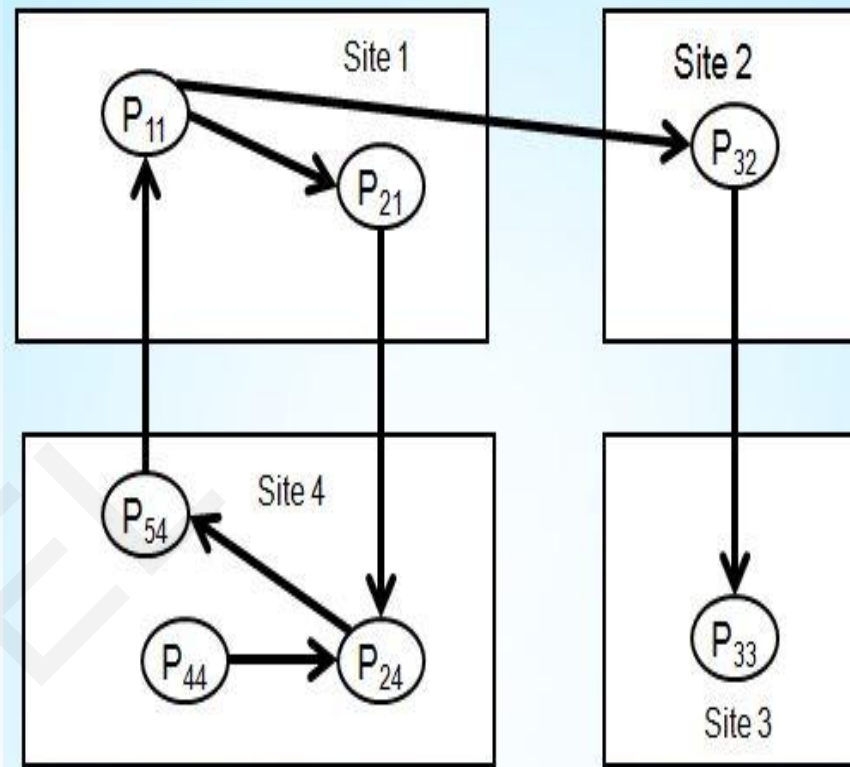
- Consider the example WFG described in the Figure 1.
- P_{11} has two outstanding resource requests. In case of the AND model, P_{11} shall become active from idle state only after both the resources are granted.
- There is a cycle $P_{11} \rightarrow P_{21} \rightarrow P_{24} \rightarrow P_{54} \rightarrow P_{11}$ which corresponds to a deadlock situation.
- That is, a process may not be a part of a cycle, it can still be deadlocked. Consider process P_{44} in Figure 1.
- It is not a part of any cycle but is still deadlocked as it is dependent on P_{24} which is deadlocked.

(iii) The OR Model

In the OR model, a process can make a request for numerous resources simultaneously and the request is satisfied if any one of the requested resources is granted.

Presence of a cycle in the WFG of an OR model does not imply a deadlock in the OR model.

- Consider example in Figure 12.1: If all nodes are OR nodes, then process P_{11} is not deadlocked because once process P_{33} releases its resources, P_{32} shall become active as one of its requests is satisfied.
- After P_{32} finishes execution and releases its resources, process P_{11} can continue with its processing.
- In the OR model, the presence of a knot indicates a deadlock.



(iv) The AND-OR Model

- A generalization of the previous two models (OR model and AND model) is the AND-OR model.
- In the AND-OR model, a request may specify any combination of *and* and *or* in the resource request.
- For example, in the AND-OR model, a request for multiple resources can be of the form $x \text{ and } (y \text{ or } z)$.
- To detect the presence of deadlocks in such a model, there is no familiar construct of graph theory using WFG.
- Since a deadlock is a stable property, a deadlock in the AND-OR model can be detected by repeated application of the test for OR-model deadlock.

(v) The $\binom{p}{q}$ Model

- Another form of the AND-OR model is the model (**called the P-out-of-Q model**), which allows a request to obtain any **k available resources** from a **pool of n resources**.
- It has the same expressive power as the **AND-OR model**.
- However, $\binom{p}{q}$ model lends itself to a much more compact formation of a request.
- Every request in the $\binom{p}{q}$ model can be expressed in the AND-OR model and vice-versa.
- Note that AND requests for p resources can be stated as $\binom{p}{p}$ and OR requests for p resources can be stated as $\binom{1}{p}$

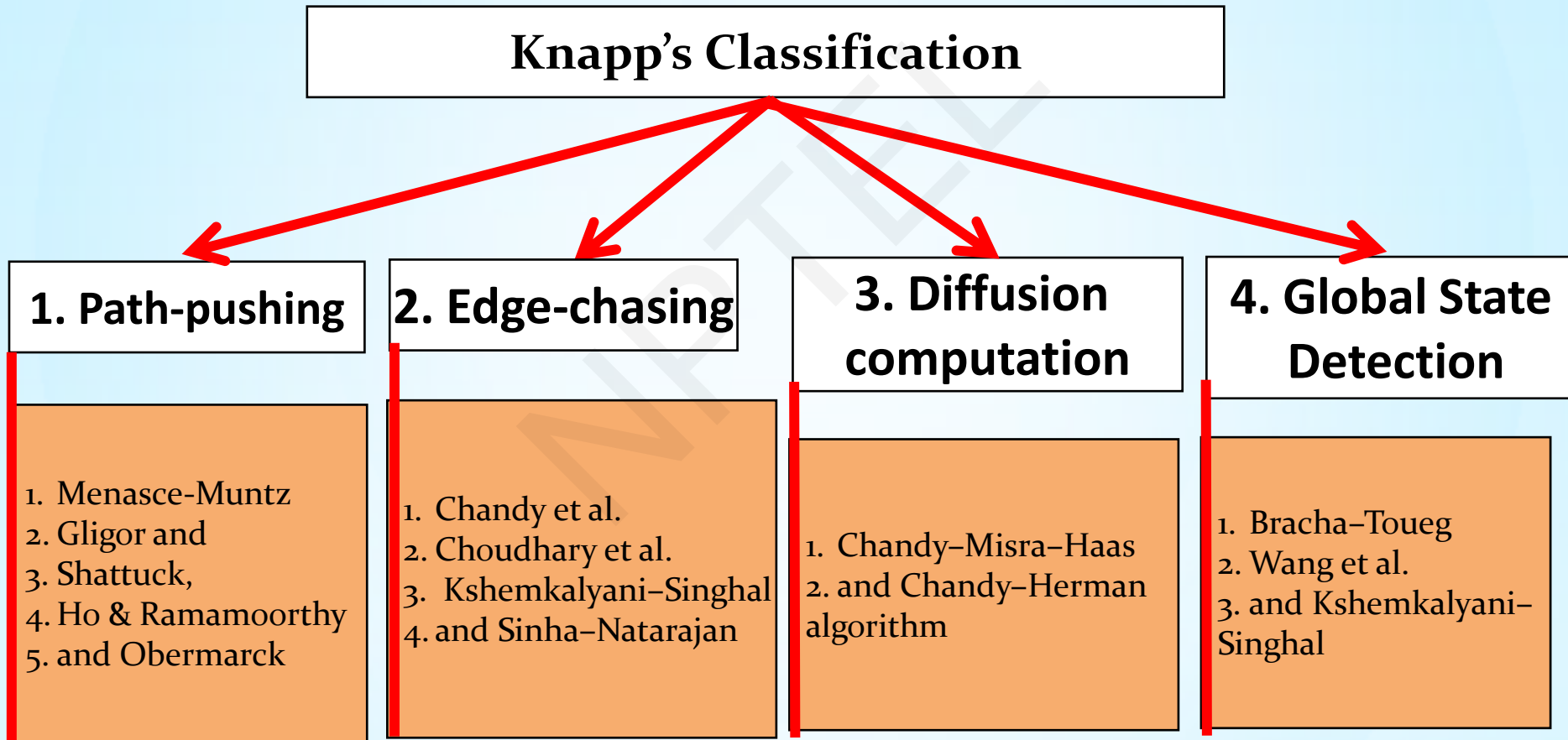
(vi) Unrestricted Model

- In the unrestricted model, no assumptions are made regarding the underlying structure of resource requests.
- Only one assumption that the deadlock is stable is made and hence it is the most general model.
- This model helps separate concerns: Concerns about properties of the problem (stability and deadlock) are separated from underlying distributed systems computations (e.g., message passing versus synchronous communication).

Classification of Distributed Deadlock Detection Algorithms

Knapp's Classification

- **E. Knapp (1987)** has classified the Distributed deadlock detection algorithms into four classes:



1. Path-Pushing Algorithms

- **In path-pushing algorithms**, distributed deadlocks are detected by maintaining an explicit global WFG.
- The basic idea is to build a global WFG for each site of the distributed system.
- In this class of algorithms, at each site whenever deadlock computation is performed, it sends its local WFG to all the neighboring sites.
- After the local data structure of each site is updated, this updated WFG is then passed along to other sites, and the procedure is repeated until some site has a sufficiently complete picture of the global state to announce deadlock or to establish that no deadlocks are present.
- This feature of sending around the paths of global WFG has led to the term path-pushing algorithms.

2. Edge-Chasing Algorithms

- **In an edge-chasing algorithm**, the presence of a cycle in a distributed graph structure is verified by propagating special messages called probes, along the edges of the graph.
- These probe messages are different than the request and reply messages.
- The formation of cycle can be detected by a site if it receives the matching probe sent by it previously.
- Whenever a process that is executing receives a probe message, it discards this message and continues.
- Only blocked processes propagate probe messages along their outgoing edges.
- Main advantage of edge-chasing algorithms is that probes are fixed size messages which is normally very short.

3. Diffusing Computations Based Algorithms

- In *diffusion computation* based distributed deadlock detection algorithms, deadlock detection computation is diffused through the WFG of the system.
- These algorithms make use of echo algorithms to detect deadlocks.
- This computation is superimposed on the underlying distributed computation. If this computation terminates, the initiator declares a deadlock.
- To detect a deadlock, a process sends out query messages along all the outgoing edges in the WFG.
- These queries are successively propagated (i.e., diffused) through the edges of the WFG.

Contd...

- When a blocked process receives first query message for a particular deadlock detection initiation, it does not send a reply message until it has received a reply message for every query it sent.
- For all subsequent queries for this deadlock detection initiation, it immediately sends back a reply message.
- The initiator of a deadlock detection detects a deadlock when it receives reply for every query it had sent out.

4. Global State Detection Based Algorithms

- Global state detection based deadlock detection algorithms exploit the following facts:
 1. A consistent snapshot of a distributed system can be obtained without freezing the underlying computation and
 2. If a stable property holds in the system before the snapshot collection is initiated, this property will still hold in the snapshot.
- Therefore, distributed deadlocks can be detected by taking a snapshot of the system and examining it for the condition of a deadlock.

Distributed Deadlock Detection Algorithms

Mitchell and Merritt's Algorithm (Edge-chasing algorithm)

- Mitchell & Merritt's algorithm (Mitchell, 1984)
 - Single-resource model
 - Detects “local” and “global” deadlocks
 - Each Process has two labels: private and public
 - Each label is (count, pid)
 - Guarantees only one process detects deadlock
- Send tokens and control information on same socket, make use of FIFO guarantee
- No synchronization mechanism required

Mitchell and Merritt's Algorithm for the Single-Resource Model

- Belongs to the class of edge-chasing algorithms where probes are sent in opposite direction of the edges of WFG.
- When a probe initiated by a process comes back to it, the process declares deadlock.
- Only one process in a cycle detects the deadlock. This simplifies the deadlock resolution – this process can abort itself to resolve the deadlock.

Contd...

- Each node of the **WFG has two local variables**, called **labels**:
 - (i) a ***private label***, which is unique to the node at all times, though it is not constant, and
 - (ii) a ***public label***, which can be read by other processes and which may not be unique.
- Each process is represented as **u/v where u and v are the public and private labels**, respectively.
- Initially, private and public labels are equal for each process.
- A global WFG is maintained and it defines the entire state of the system.

Contd...

- The algorithm is defined by the four state transitions shown in **Figure 12.2**, where **$z = \text{inc}(u, v)$** , and **$\text{inc}(u, v)$ yields a unique label** greater than both u and v labels that are not shown do not change.
- Block creates an edge in the WFG.
- Two messages are needed, one resource request and one message back to the blocked process to inform it of the public label of the process it is waiting for.
- Activate denotes that a process has acquired the resource from the process it was waiting for.
- Transmit propagates larger labels in the opposite direction of the edges by sending a probe message.

Possible State Transitions

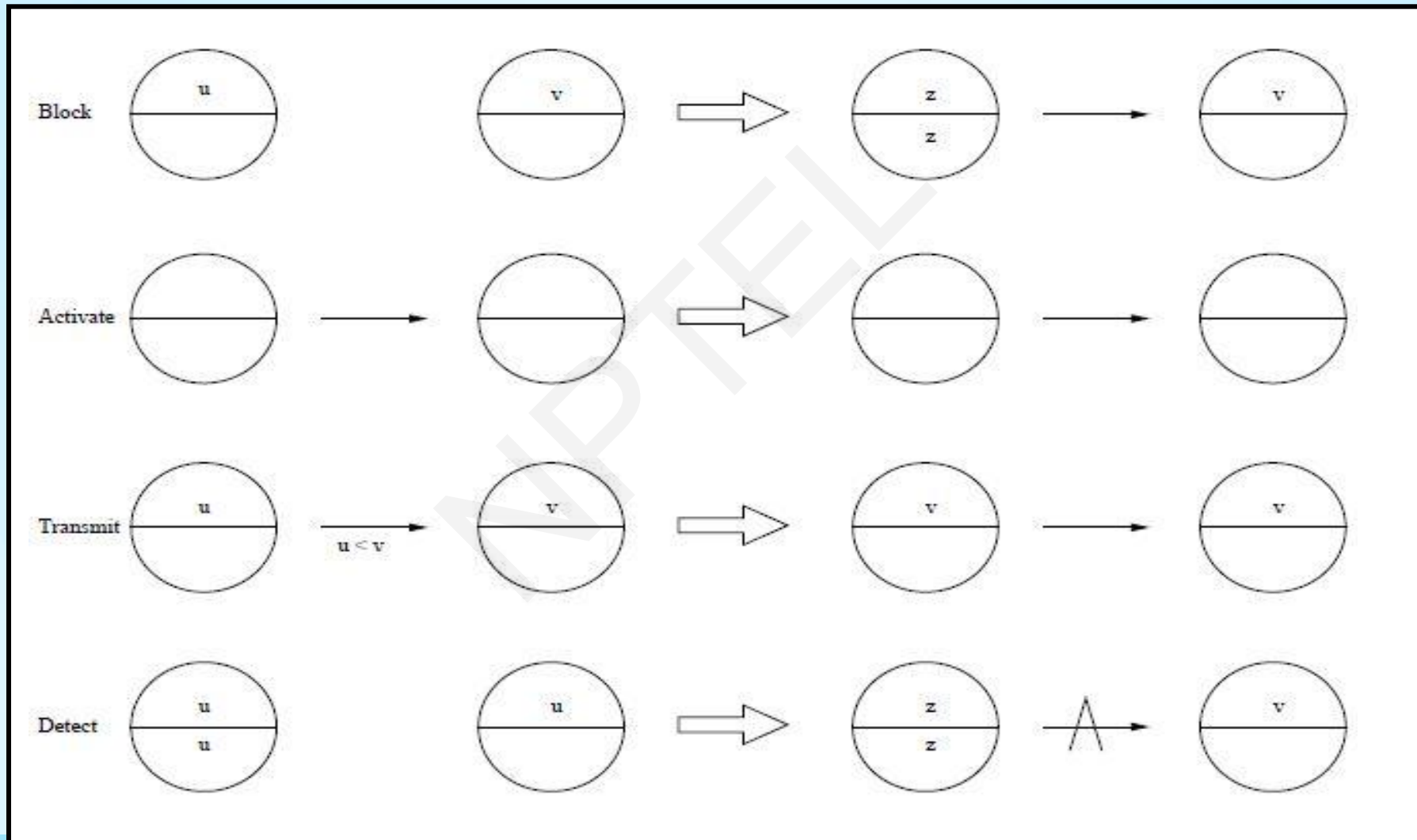


Figure 12.2: The four possible state transitions

Contd...

- Whenever a process receives a probe which is less than its public label, then it simply ignores that probe.
- Detect means that the probe with the private label of some process has returned to it, indicating a deadlock.
- The above algorithm can be easily extended to include priorities where whenever a deadlock occurs, the lowest priority process gets aborted.

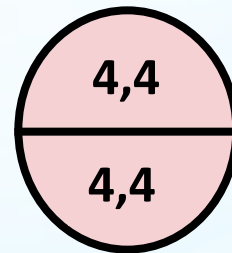
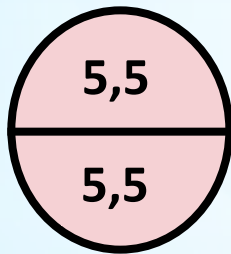
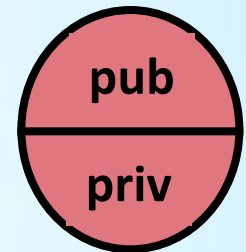
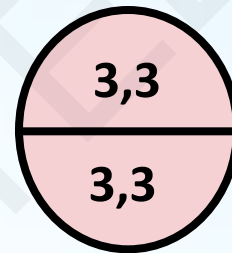
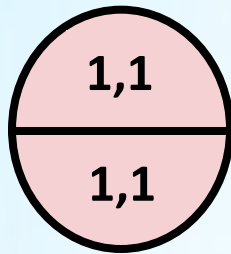
Algorithm

1. Every node has a **public and private label**; both are non-decreasing. Additionally, no two nodes ever have the same private label.
2. When **node x** begins to wait on **node y**, node x updates its the public label to be **$\max(x, y) + 1$** .
3. When a node discovers that the node it is waiting on has a larger public label than its own, it replaces the value of its public label with the larger one.
4. This algorithm has the effect of circulating successively larger public labels in the reverse order of the corresponding wait-for graph. If a deadlock truly exists, then a node will eventually see its own public label on the process for which it waits.

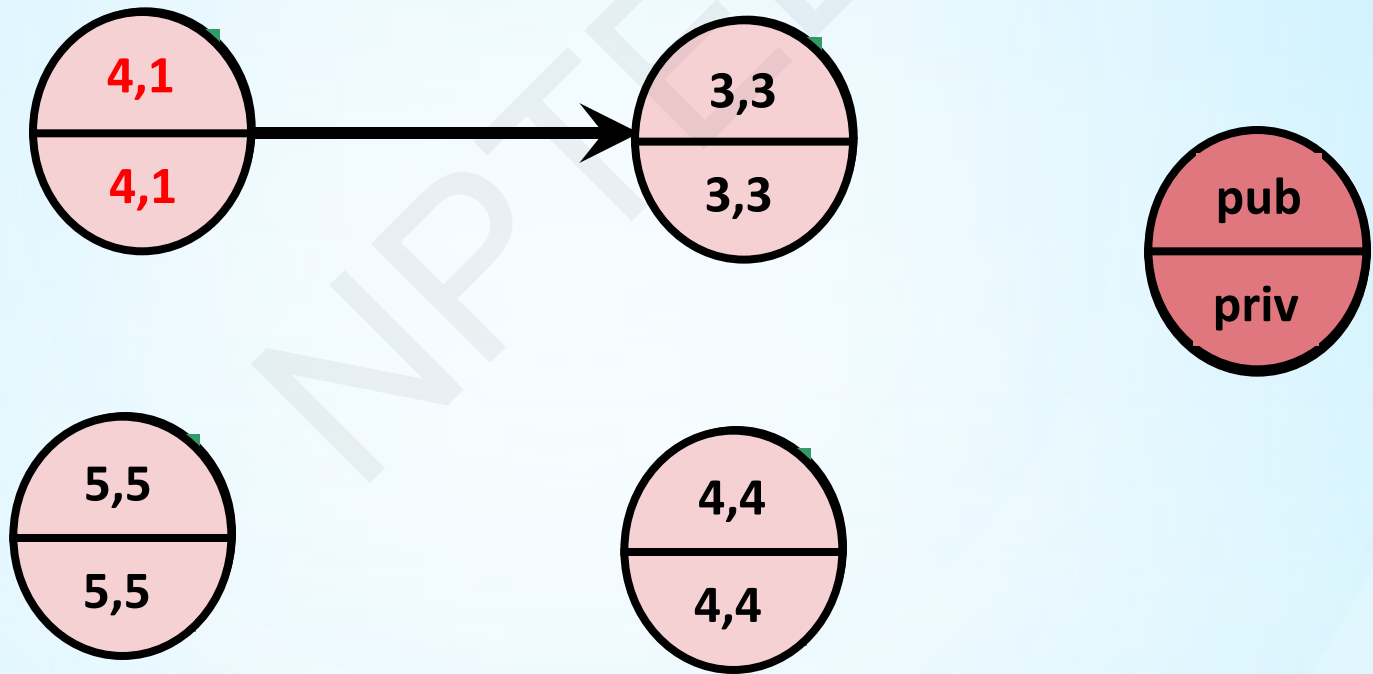
Important Property

- This algorithm also has the nice property that exactly one node will detect a deadlock.
- This property is important for deadlock resolution in process networks, as it helps to ensure that exactly one channel's capacity is increased.
- In addition, it is trivial to modify this algorithm so that upon detection of deadlock would reveal the smallest channel capacity.

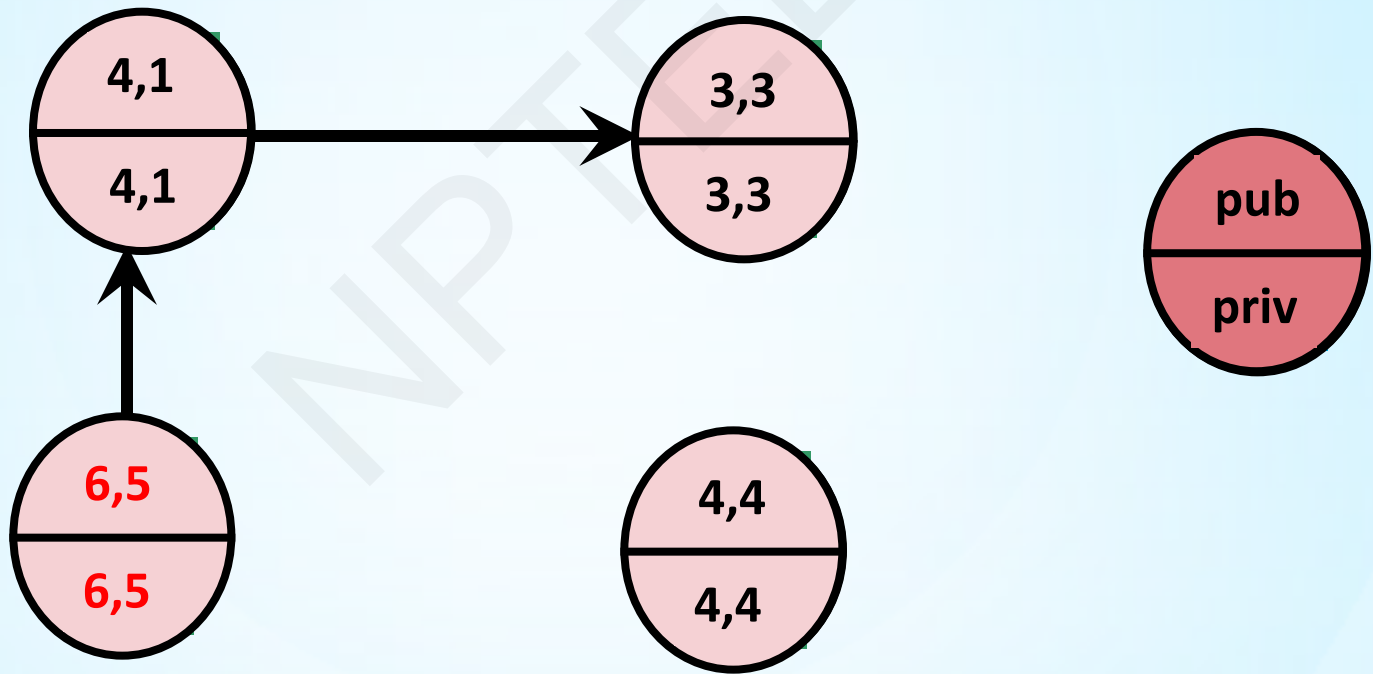
Mitchell-Merritt Example



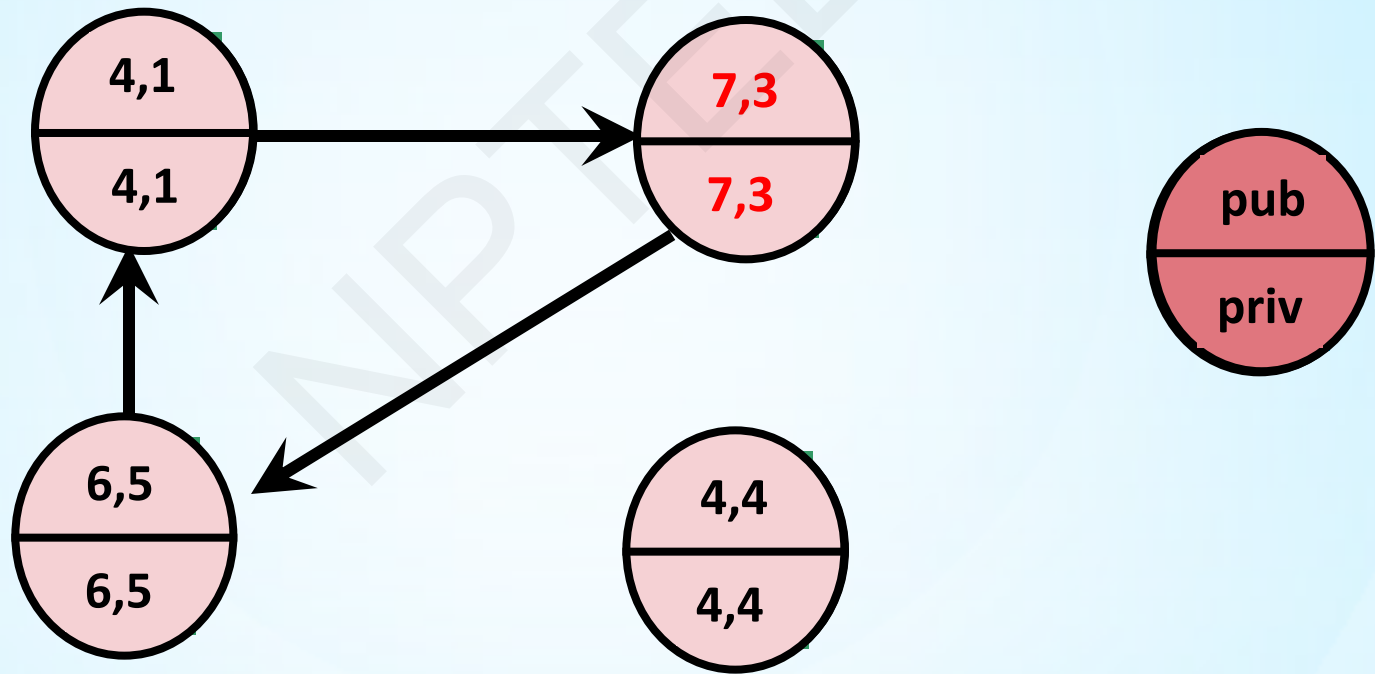
BLOCK



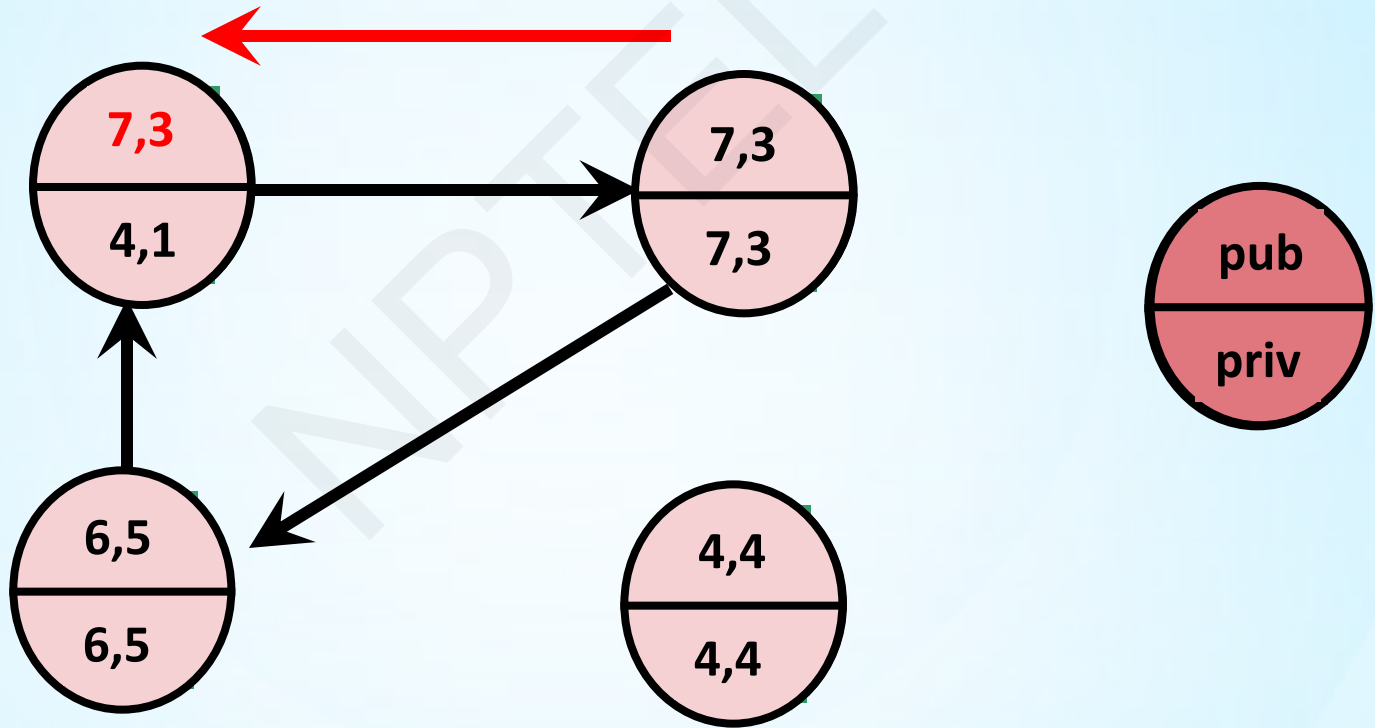
BLOCK



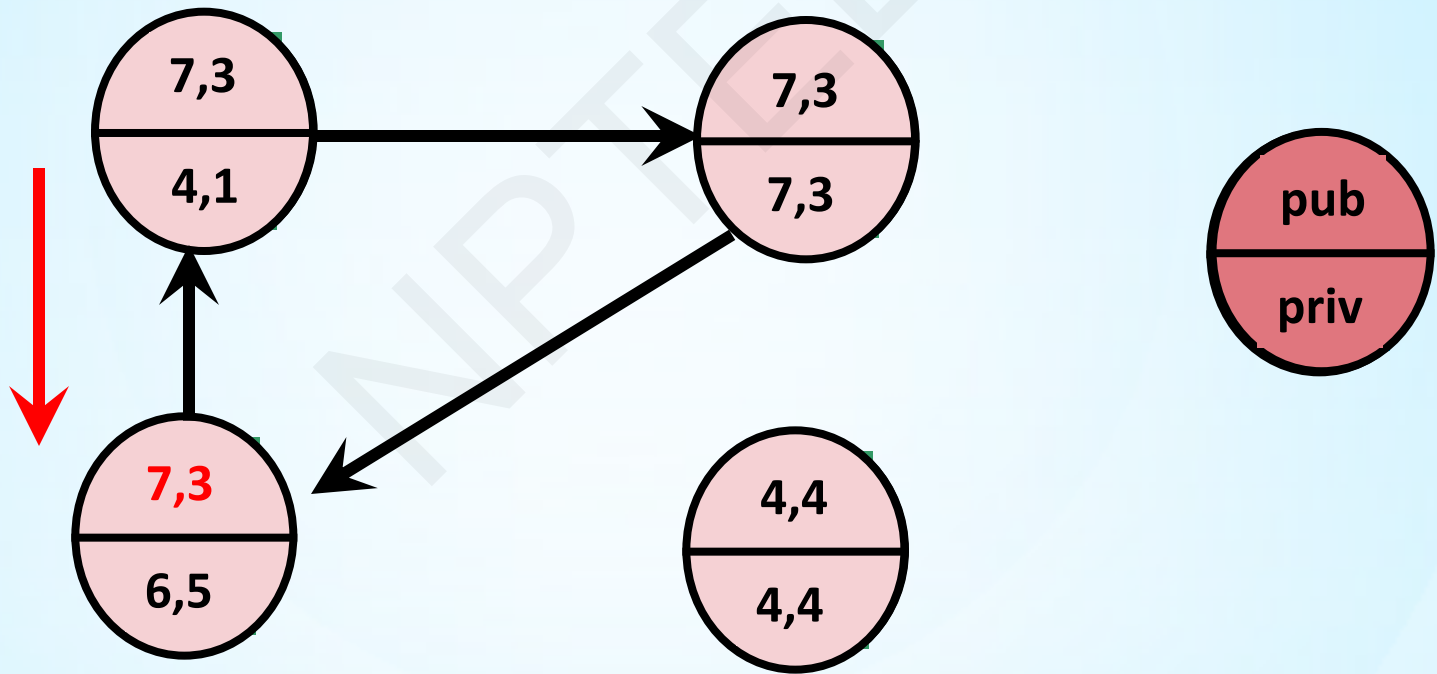
BLOCK



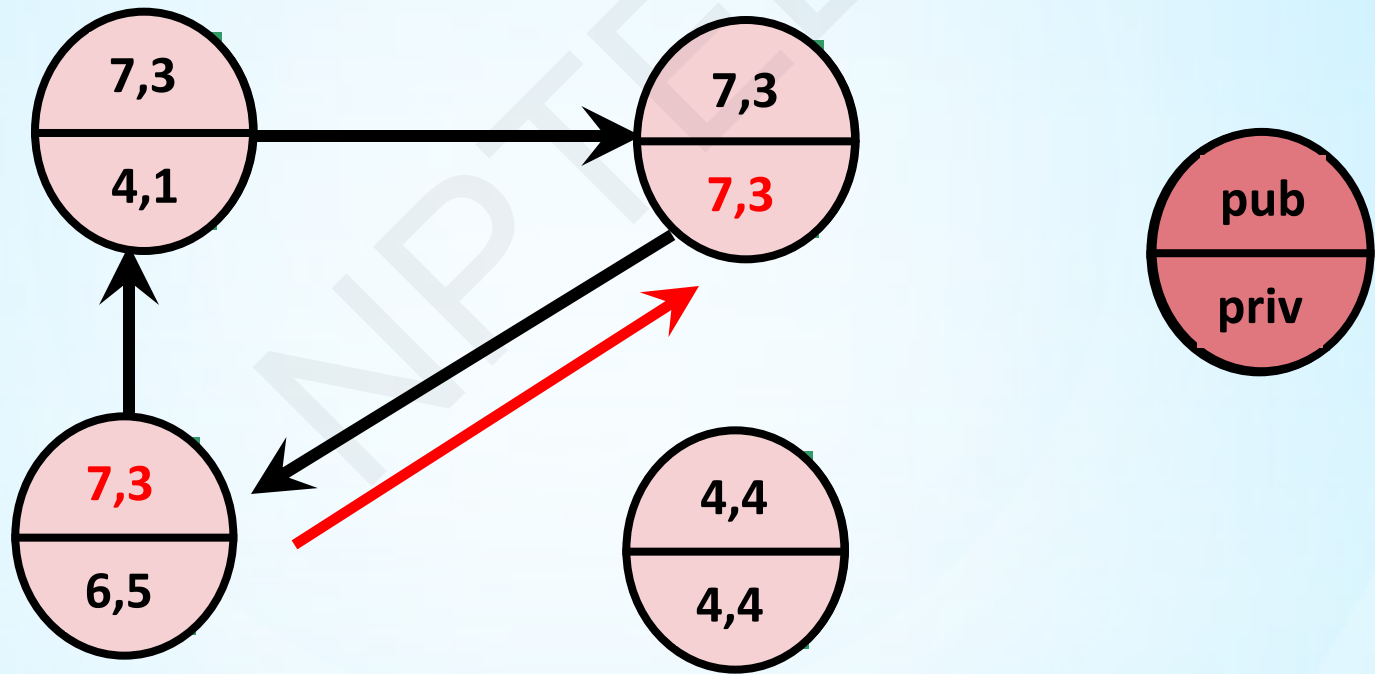
Transmit



Transmit



DEADLOCK



Message Complexity

If we assume that a deadlock persists long enough to be detected, the **worst-case complexity of the algorithm is:**

$s(s - 1)/2$ Transmit steps,

where **s is the number of processes** in the cycle.

Few Other Algorithms for Distributed Deadlock Detection

Author	Year	Class	Algorithm
Menasce and Muntz	1979	Path-Pushing	Locking and deadlock detection in distributed databases
Gligor and Shattuck	1980	Path-Pushing	Deadlock detection in distributed databases
Ho and Ramamoorthy	1982	Path-Pushing	Protocols for deadlock detection in distributed
Obermarck	1982	Path-Pushing	Distributed deadlock detection algorithm
Chandy–Misra–Haas	1983	Edge-chasing	Distributed deadlock detection algorithm for the AND model
Choudary et al.	1989	Edge-chasing	Modified priority based probe algorithm for distributed deadlock detection and resolution
Chandy–Misra–Haas	1983	Diffusion Computation	Distributed deadlock detection algorithm for the OR model
Kshemkalyani–Singhal	1994	Global state detection approach.	To detect deadlocks in the P-out-of-Q model (also called the generalized distributed deadlocks)

Conclusion

- Out of the three approaches to handle deadlocks, **deadlock detection is the most promising in distributed systems**. Detection of deadlocks requires performing two tasks: first, maintaining or constructing whenever needed a WFG; second, searching the WFG for a deadlock condition (cycles or knots).
- Distributed deadlock detection algorithms can be divided into four classes: (i) path-pushing, (ii) edge-chasing, (iii) diffusion computation, and (iv) global state detection.
- In this lecture we have discussed one algorithm i.e. Mitchell and Merritt's Algorithm for the Single-Resource Model.
- In upcoming lecture, we will discuss about '**Distributed Shared Memory**'.

Lecture: 13

Distributed Shared Memory



Rajiv Misra

Dept. of Computer Science &
Engineering

Indian Institute of Technology Patna

rajivm@iitp.ac.in

Preface

Recap of Previous Lecture:

- In the previous lecture, we have discussed the **basic fundamentals of 'Distributed Deadlock Detection'** and
- **Different classes of algorithms** such as path-pushing, edge-chasing, diffusion computation, and global state detection.

Content of this Lecture:

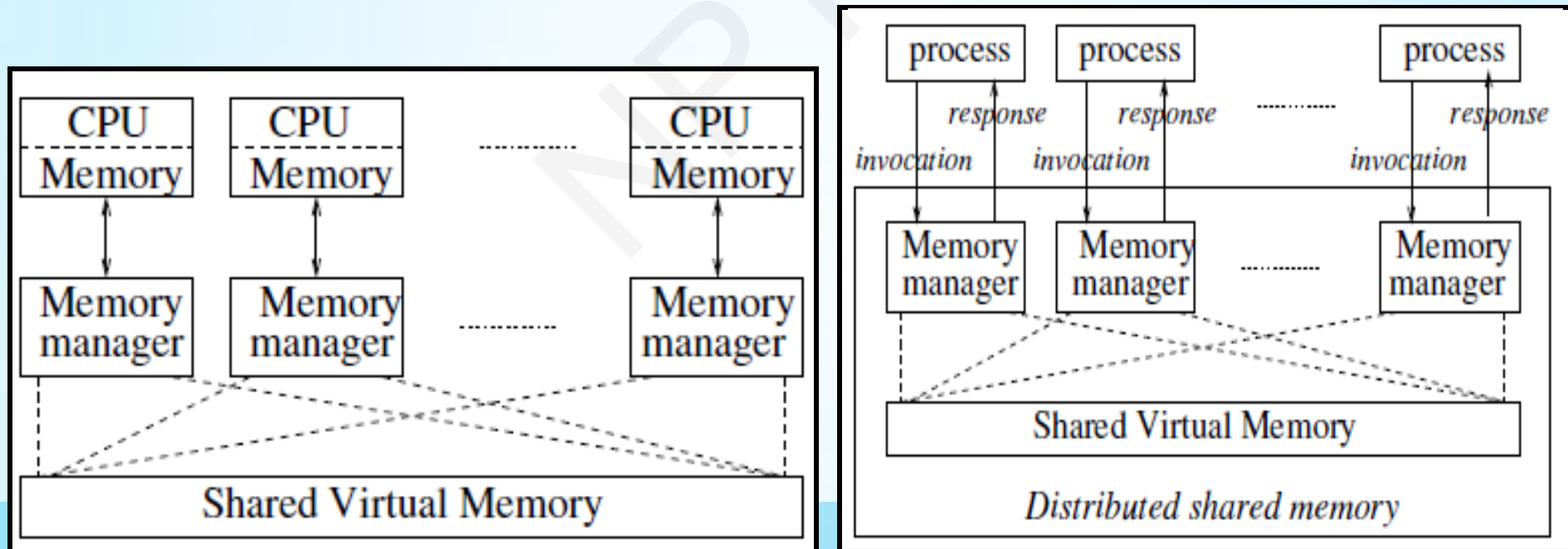
- In this lecture, we will discuss about the concept of **'Distributed Shared Memory (DSM)'** as well as provide different ways to classify DSMs and **'consistency models'** of DSM and
- Also Discuss **'Lamport's Bakery Algorithm'** for Shared Memory Mutual Exclusion.

Introduction

- **Distributed shared memory (DSM)** is an abstraction provided to the programmer of a distributed system. It gives the impression of a single monolithic memory, as in traditional von Neumann architecture.
- Programmers access the data across the network using only *read* and *write* primitives, as they would in a uniprocessor system.
- Programmers do not have to deal with *send* and *receive* communication primitives and the ensuing complexity of dealing explicitly with synchronization and consistency in the message passing model.

Distributed Shared Memory Abstractions

- Communicate with **Read/Write** operations in **shared virtual space**
- No Send and Receive primitives to be used by application
 - Under covers, Send and Receive used by DSM manager
- Locking is too restrictive; need concurrent access
- With **replica management**, problem of consistency arises!
- $\Rightarrow$ weaker consistency models (weaker than von Neumann) required



Advantages of DSM

1. Shields programmer from Send/Receive primitives
2. Single address space; simplifies *passing-by-reference* and passing complex data structures
3. Exploit *locality-of-reference* when a block is moved
4. DSM uses simpler software interfaces, and cheaper off-the-shelf hardware. Hence cheaper than dedicated multiprocessor systems
5. No memory access bottleneck, as no single bus Large virtual memory space
6. DSM programs portable as they use common DSM programming interface.

Disadvantages of DSM

1. Programmers **need to understand consistency models**, to write correct programs
2. DSM implementations **use asynchronous message-passing**, and hence cannot be more efficient than message-passing implementations
3. By yielding **control to DSM manager software**, programmers cannot use their own message-passing solutions.

Issues in Implementing DSM Software

- **Semantics for concurrent access** must be clearly specified
- **Semantics—replication? partial? full? read-only? write-only?**
- **Locations** for replication (for optimization)
- If not full replication, determine **location of nearest data** for access
- **Reduce delays, no. of messages** to implement the semantics of concurrent access
- Data is **replicated or cached**
- **Remote access** by Hardware or Software
- **Caching/replication** controlled by Hardware or Software
- DSM **controlled** by memory management Software, OS, language run-time system

Comparison of Early DSM Systems

Type of DSM	Examples	Management	Caching	Remote access
Single-bus multiprocessor	Firefly, Sequent	by MMU	hardware control	by hardware
Switched multiprocessor	Alewife, Dash	by MMU	hardware control	by hardware
NUMA system	Butterfly, CM	by OS	software control	by hardware
Page-based DSM	Ivy, Mirage	by OS	software control	by software
Shared variable DSM	Midway, Munin	by language runtime system	software control	by software
Shared object DSM	Linda, Orca	by language runtime system	software control	by software

Memory Consistency Models

Memory Coherence

- **Memory coherence** is the ability of the system to execute memory operations correctly.
- Assume **n processes** and **s_i memory operations** by per **process P_i**
- Also assume that all the operations issued by a process are executed sequentially (that is, pipelining is disallowed), as shown in figure 13.3.

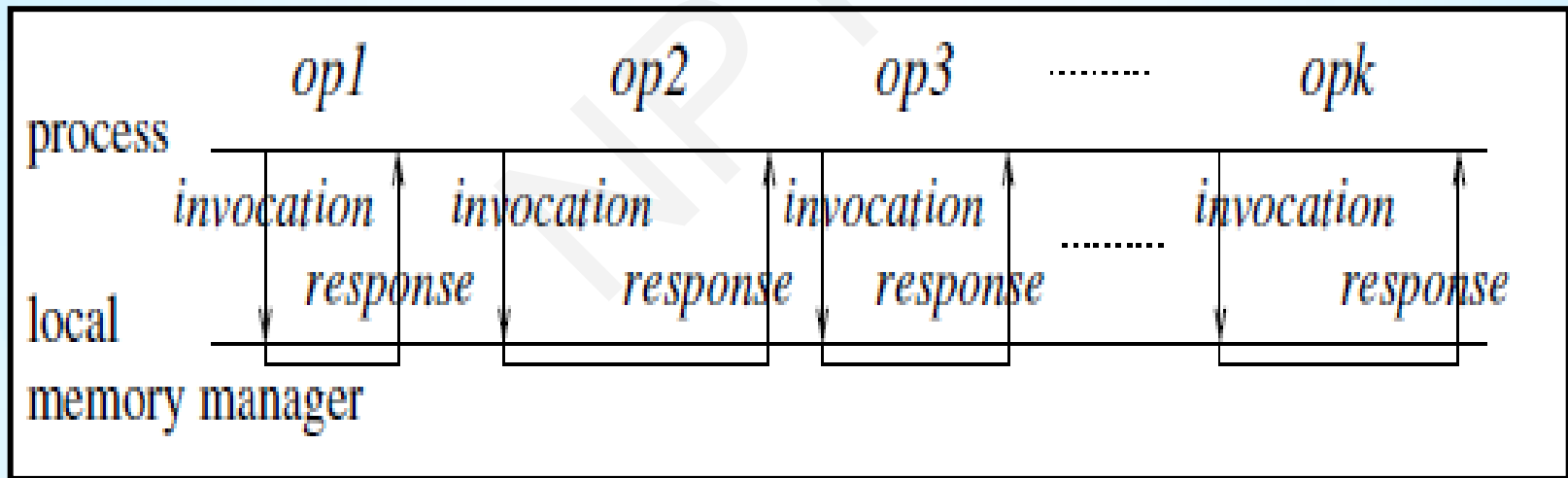


Figure 13.3 Sequential invocations and responses in a DSM system, without any pipelining.

Memory Coherence

- Observe that there are a total of

$(s_1 + s_2 + \dots s_n)! / (s_1! s_2! \dots s_n!)$ possible interleavings

- Memory coherence model defines which interleavings are permitted.
- Traditionally, Read returns the value written by the most recent Write
- **"Most recent"** Write is ambiguous with replicas and concurrent accesses
- DSM consistency model is a **contract** between DSM system and application programmer

Consistency Models

Author	Year	Consistency Model
L. Lamport	1979	Sequential consistency model
L. Lamport	1986	Linearizability model
Lipton and Sandberg	1988	PRAM model
M. Herlihy and J. Wing	1990	Linearizability model
P. Hutto and M. Ahamad	1990	Slow memory model
M. Dubois and C. Scheurich	1990	Weak consistency model
K. Gharachorloo et al.	1990	Release consistency model
H. Bal et al.	1992	Sequential consistency model
B. Bershad et al.	1993	Entry consistency model
H. Attiya and J. Welch	1994	Linearizability and Sequential consistency
M. Ahamad et al.	1995	Causal consistency model

1. Strict Consistency/Linearizability/Atomic Consistency

Strict consistency (SC)

1. Any **Read** to a location (variable) is required to return the value written by the most recent **Write** to that location (variable) as per a global time reference.
2. All operations appear to be executed atomically and sequentially.
3. All processors see the same ordering of events, which is equivalent to the global-time occurrence of non-overlapping events.

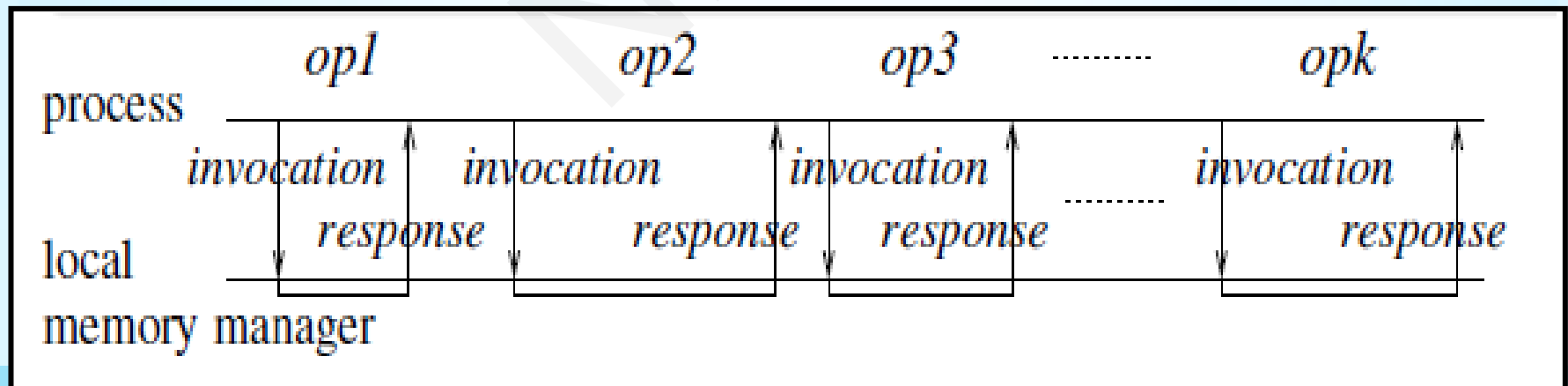


Fig 13.4 Sequential invocations and responses to each Read or Write operation.

Conditions of Linearizability

- More formally, a sequence Seq of invocations and responses is **linearizable (LIN)** if there is a permutation Seq' of adjacent pairs of corresponding $\langle invoc, resp \rangle$ events satisfying:
 - C1.** For every variable v , the projection of Seq' on v , denoted Seq'_v , is such that **every Read (adjacent $\langle invoc, resp \rangle$ event pair) returns the most recent Write (adjacent $\langle invoc, resp \rangle$ event pair) that immediately preceded it.**
 - C2.** If the response $op1(resp)$ of operation $op1$ occurred before the invocation $op2(invoc)$ of operation $op2$ in Seq , then $op1$ (adjacent $\langle invoc, resp \rangle$ event pair) occurs before $op2$ (adjacent $\langle invoc, resp \rangle$ event pair) in Seq' .
- **Condition 1** specifies that every processor sees a common order Seq' of events, and that in this order, the semantics is that **each Read returns the most recent completed Write value. Condition 2** specifies that the **common order Seq' must satisfy the global time order of events, viz.,** the order of non-overlapping operations in Seq must be preserved in Seq' .

Strict Consistency/Linearizability: Examples

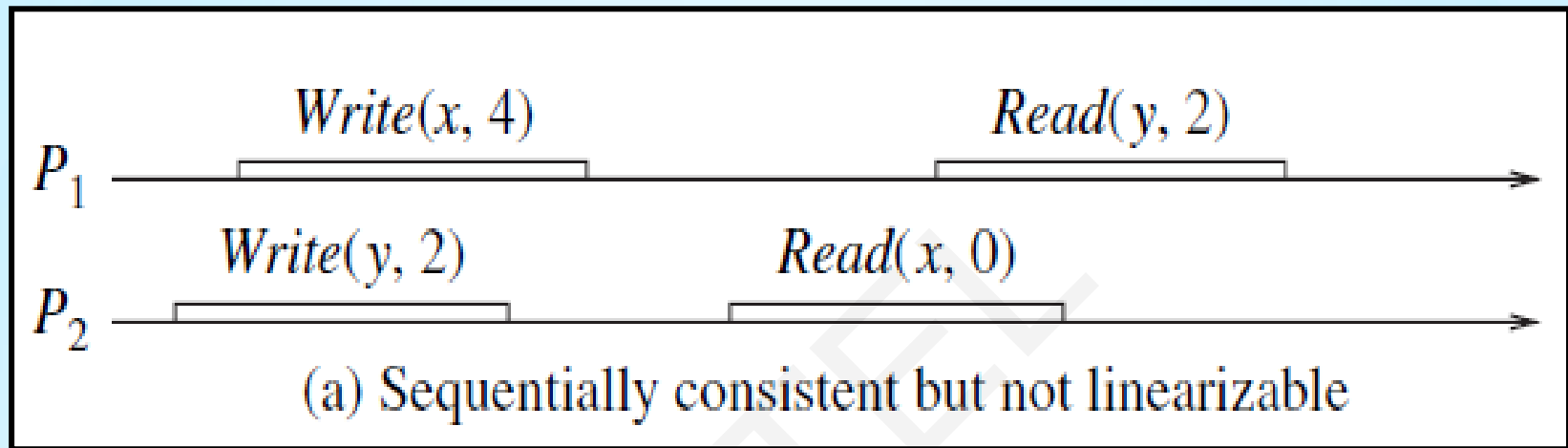


Figure 13.5(a) The execution is not linearizable because although the Read by P_2 begins after Write (x, 4), the Read returns the value that existed before the Write. Hence, a permutation Seq' satisfying the condition 2 above on global time order does not exist.

Contd...

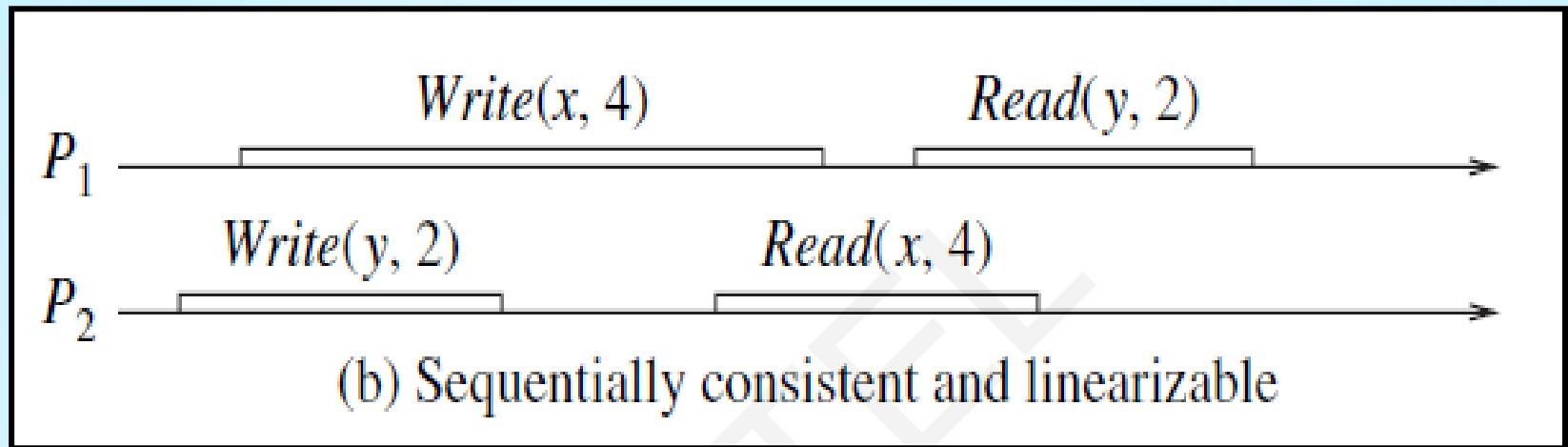


Figure 13.5(b) The execution is linearizable. The global order of operations (corresponding to $\langle \text{invocation}, \text{response} \rangle$ pairs in Seq'), consistent with the real-time occurrence, is: $Write(y, 2)$, $Write(x, 4)$, $Read(x, 4)$, $Read(y, 2)$. This permutation Seq' satisfies conditions 1 and 2.

Contd...

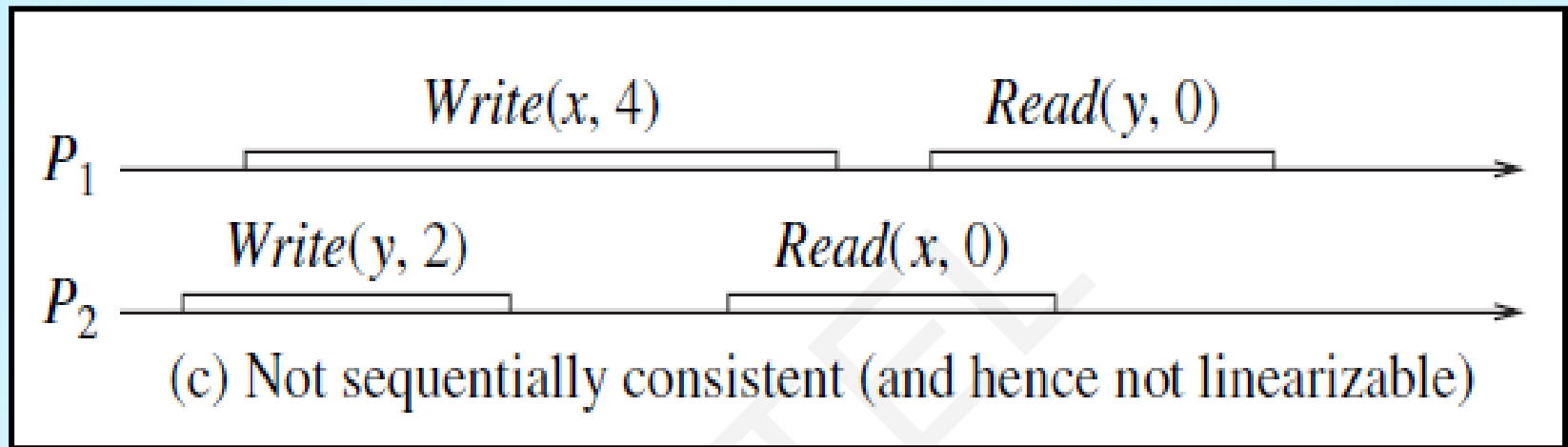


Figure 13.5(c) The execution is not linearizable. The two dependencies: $Read(x, 0)$ before $Write(x, 4)$, and $Read(y, 0)$ before $Write(y, 2)$ cannot both be satisfied in a global order while satisfying the local order of operations at each processor. Hence, there does not exist any permutation Seq' satisfying conditions 1 and 2.

Linearizability: Implementation

- Simulating global time axis is expensive.
- Assume full replication, and total order broadcast support.

(shared var)

int: *x* ;

- (1) When the Memory Manager receives a *Read* or *Write* from application:
 - (1a) **total_order_broadcast** the *Read* or *Write* request to all processors;
 - (1b) **await** own request that was broadcast;
 - (1c) **perform** pending response to the application as follows
 - (1d) **case** *Read*: return value from local replica;
 - (1e) **case** *Write*: write to local replica and return ack to application.
- (2) When the Memory Manager receives a **total_order_broadcast**(*Write*, *x*, *val*) from network:
 - (2a) **write** *val* to local replica of *x* .
- (3) When the Memory Manager receives a **total_order_broadcast** (*Read*, *x*) from network:
 - (3a) **no operation**.

Algorithm 13.1 Implementing linearizability (LIN) using total order broadcasts .

Code shown is for P_i , $1 \leq i \leq n$.

Linearizability: Implementation (2)

- For a Read operation, when the memory managers system wide receive the total order broadcast, they do not perform any action.

- **Why is the broadcast then necessary?**

- The reason is this. If Read operations do not participate in the total order broadcasts, they do not get totally ordered with respect to the Write operations as well as with respect to the other Read operations.

- This can lead to a violation of linearizability as shown in **Figure 13.6**

Linearizability: Implementation (2)

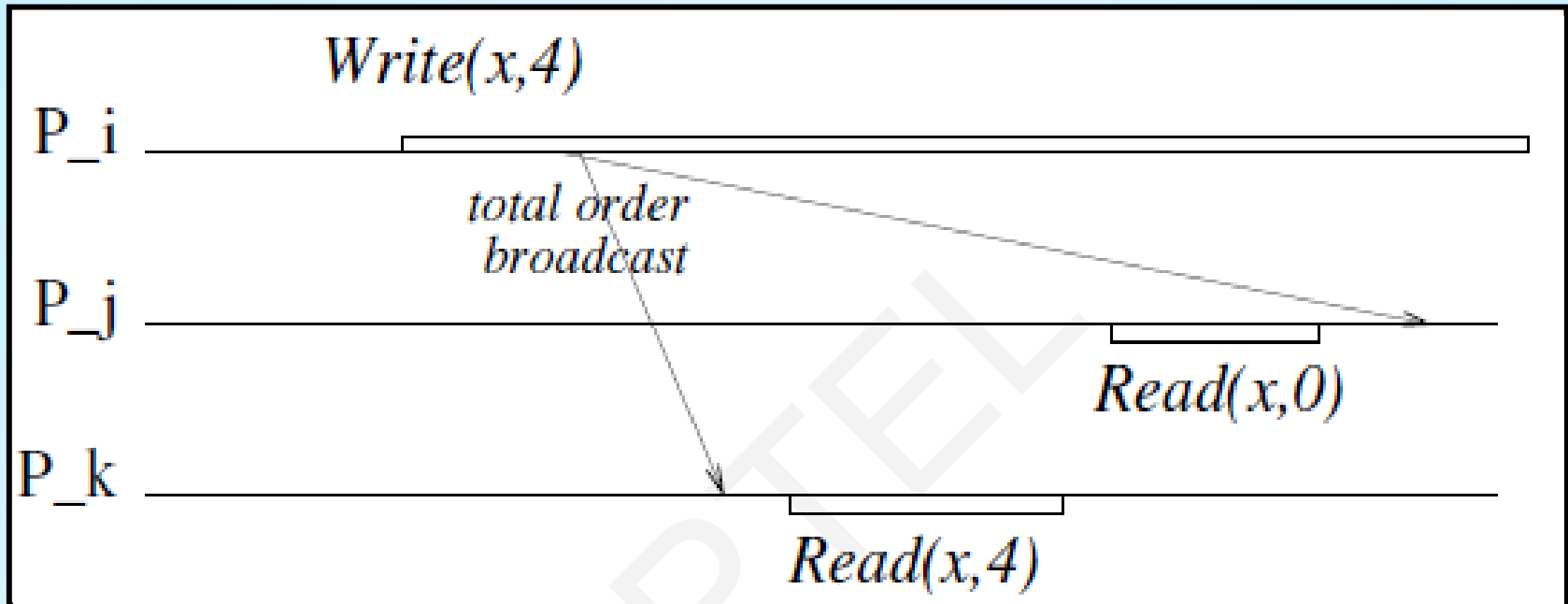


Figure 13.6 A violation of linearizability (LIN) if Read operations do not participate in the total order broadcast

The **Read** by P_k returns the value written by P_i . The later *Read* by P_j **returns the initial** value of 0. As per the global time ordering requirement of linearizability, the **Read** by P_j that occurs after the **Read** by P_k must also return the value 4. However, that is not the case in this example, wherein the **Read operations** do not participate in the total order broadcast.

2. Sequential Consistency

Sequential consistency is specified as follows:

- The result of any execution is the same as if all operations of the processors were executed in *some sequential order*.
- The operations of each individual processor appear in this sequence in the *local program order*.
- Any interleaving of the operations from the different processors is possible. But all processors must see *the same* interleaving. Even if two operations from different processors (on the same or different variables) do not overlap in a global time scale, they may appear in reverse order in the *common* sequential order seen by all the processors. See examples used for linearizability.

Contd...

Only Writes participate in total order broadcasts. Reads do not because:

- all consecutive operations by the same processor are ordered in that same order (no pipelining), and
- Read** operations by different processors are independent of each other; to be ordered only with respect to the **Write** operations.
- Direct simplification of the linearizability (LIN) algorithm.
- Reads executed atomically. Not so for Writes.
- Suitable for Read-intensive programs.

Sequential Consistency using Local Reads

(shared var)

int: x ;

(1) When the Memory Manager at P_i receives a **Read** or **Write** from application:

(1a) **case Read**: **return** value from local replica;

(1b) **case Write(x, val)**: **total_order_broadcast_j**(**Write(x, val)**) to all processors including itself.

(2) When the Memory Manager at P_i receives a **total_order_broadcast_j**(**Write, x, val**) from network

(2a) **write** val to local replica of x ;

(2b) **if** $i = j$ **then return** acknowledgement to application.

Algorithm 13.2 Implementing sequential consistency (SC) using local Read operations . Code shown is for P_i , $1 \leq i \leq n$.

Sequential Consistency using Local Writes

(shared var)

int: x ;

(1) When the Memory Manager at P_i receives a $Read(x)$ from application:

(1a) **if** $counter = 0$ **then**

(1b) **return** x

(1c) **else** Keep the $Read$ pending.

(2) When the Memory Manager at P_i receives a $Write(x, val)$ from application:

(2a) $counter \leftarrow counter + 1$;

(2b) **total_order_broadcast**; the $Write(x, val)$;

(2c) **return** acknowledgement to the application.

(3) When the Memory Manager at P_i receives a **total order broadcast**; ($Write, x, val$) from network

(3a) **write** val to local replica of x .

(3b) **if** $i = j$ **then**

(3c) $counter \leftarrow counter - 1$;

(3d) **if** ($counter = 0$ and any $Reads$ are pending) **then**

(3e) **perform** pending responses for the $Reads$ to the application.

Algorithm 13.3 Locally issued Writes get acked immediately. Local Reads are delayed until the locally preceding Writes have been acked. All locally issued Writes are pipelined.

3. Causal Consistency

In SC, all Write ops should be seen in common order.

For **causal consistency**, only causally related Writes should be seen in common order.

Causal relation for shared memory systems:

- At a processor, local order of events is the causal order
- A Write causally precedes Read issued by another processor if the Read returns the value written by the Write.
- The transitive closure of the above two orders is the causal order

Total order broadcasts (for SC) also provide causal order in shared memory

Examples

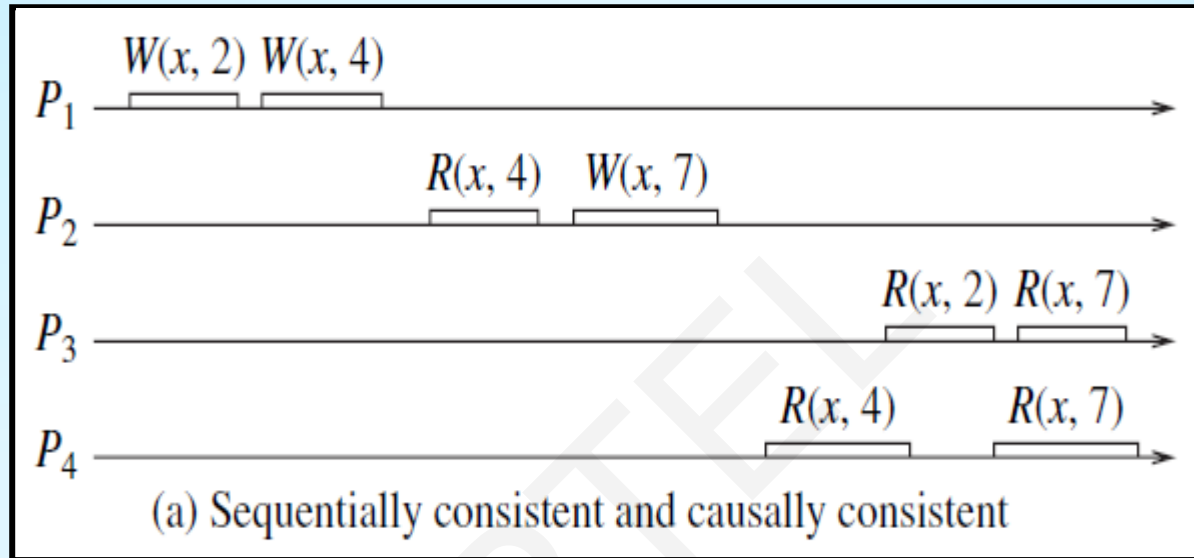


Figure 13.7(a) The execution is sequentially consistent (and hence causally consistent). Both P_3 and P_4 see the operations at P_1 and P_2 in sequential order and in causal order.

Contd...

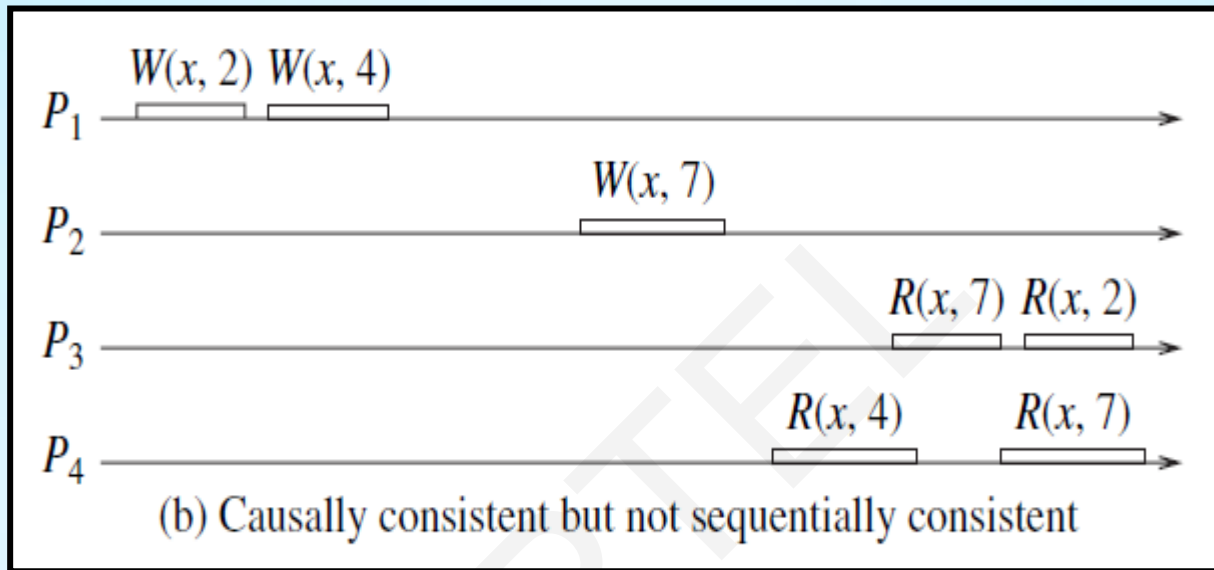


Figure 13.7(b) The execution is not sequentially consistent but it is causally consistent. Both P_3 and P_4 see the operations at P_1 and P_2 in causal order because the lack of a causality relation between the *Writes by P_1 and by P_2* allows the values written by the two processors to be seen in different orders in the system. The execution is not sequentially consistent because there is no global satisfying the contradictory ordering requirements set by the *Reads by P_3 and by P_4* .

Contd...

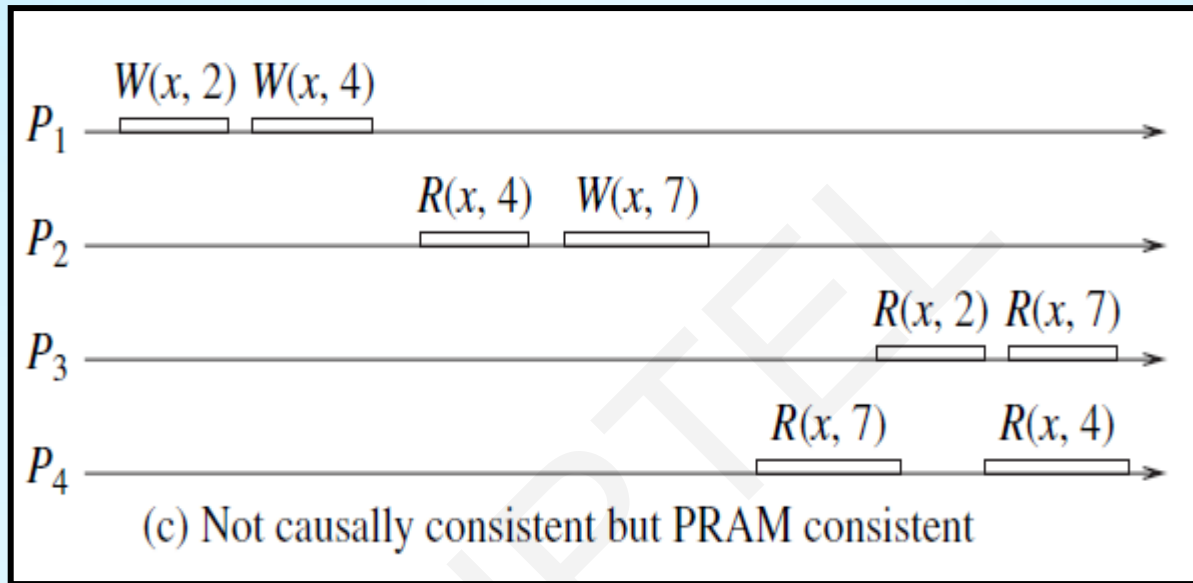


Figure 13.7(c) The execution is not causally consistent because **the second** Read by P_4 returns 4 after P_4 has already returned 7 in an earlier Read.

4. Pipelined RAM or Processor Consistency

PRAM memory

- Only Write operations issued by the same processor are seen by others in the order they were issued, but Writes from different processors may be seen by other processors in different orders.
- PRAM can be implemented by FIFO broadcast? PRAM memory can exhibit counter-intuitive behavior, see below:

(shared variables)

int: x, y ;

Process 1

...

(1a) $x \leftarrow 4$;

(1b) if $y = 0$ then kill(P_2).

Process 2

...

(2a) $y \leftarrow 6$;

(2b) if $x = 0$ then kill(P_1).

Algorithm 13.4 A counter-intuitive behavior of a PRAM-consistent program.

The initial values of variables are zero.

5. Slow Memory

- Only Write operations issued by the same processor and to the same memory location must be seen by others in that order.

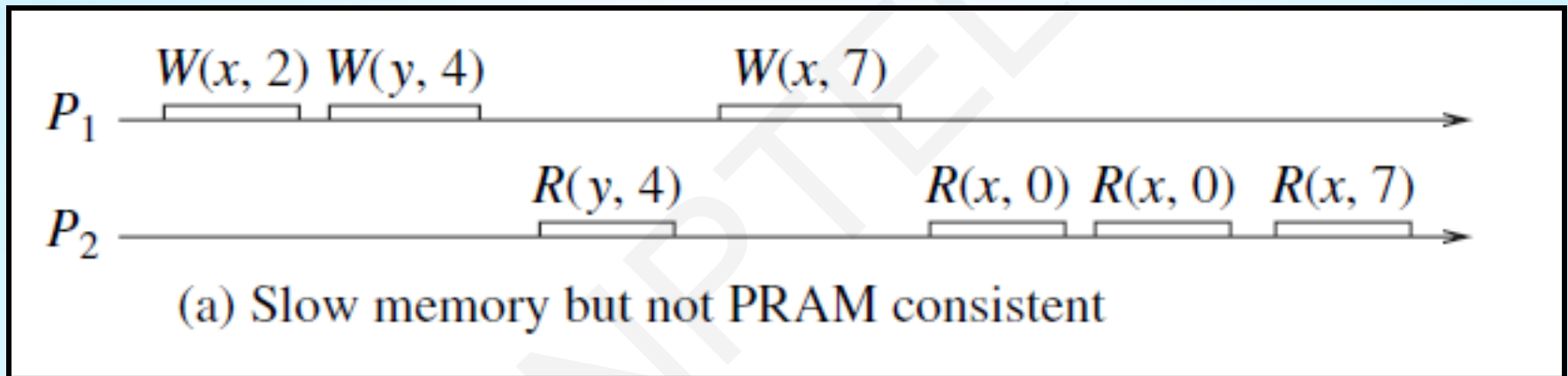


Figure 13.8(a) The updates to each of the variables are seen pipelined separately in a FIFO fashion. The “x” pipeline from P_1 to P_2 is slower than the “y” pipeline from P_1 to P_2 . Thus, the overtaking effect is allowed. However, PRAM consistency is violated because the FIFO property is violated over the single common “pipeline” from P_1 to P_2 – the update to y is seen by P_2 but the much older value of $x = 0$ is seen by P_2 later.

Contd...

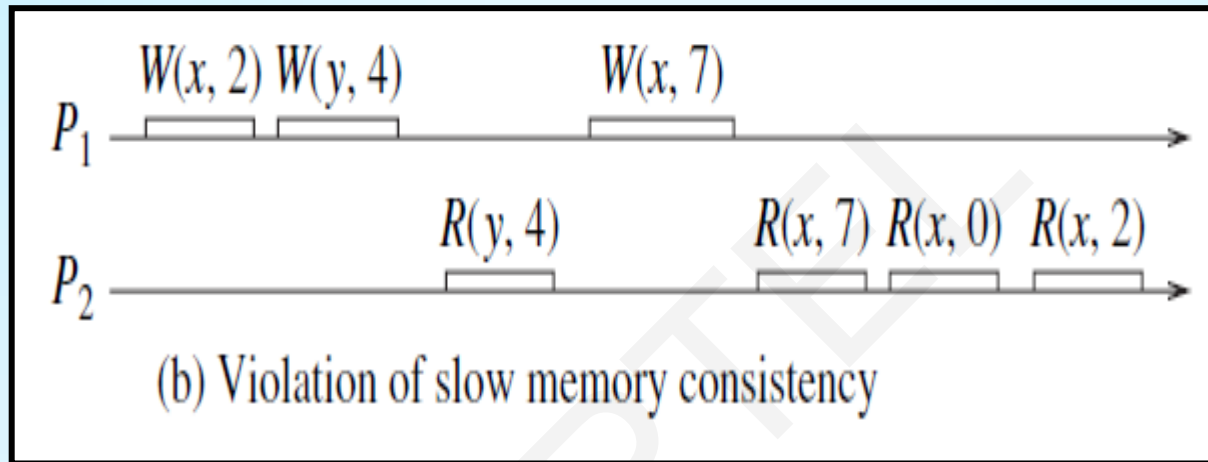
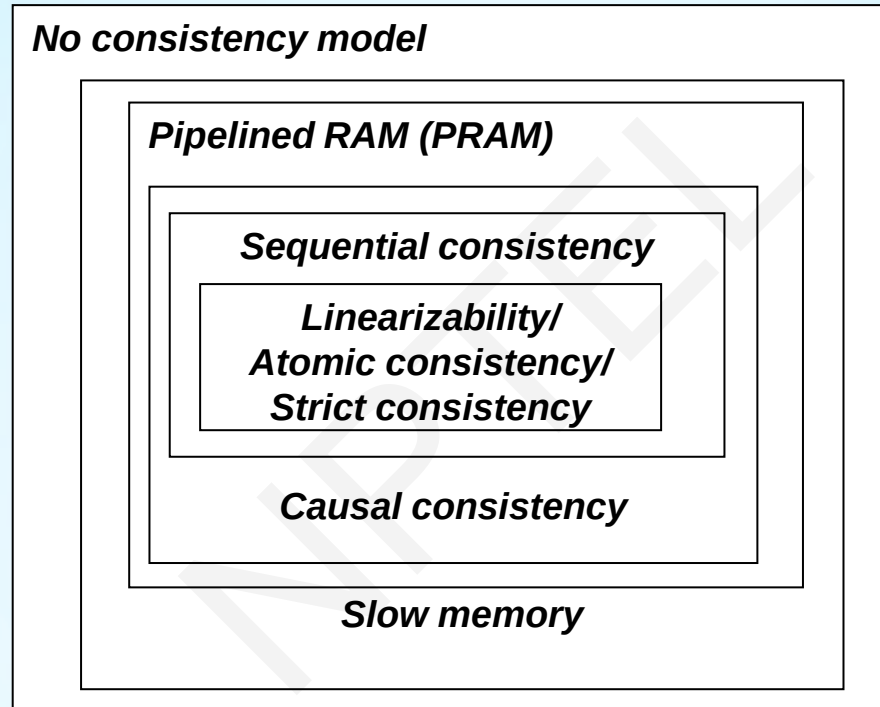


Figure 13.8(b) Slow memory consistency is violated because the FIFO property is violated for the pipeline for variable x . “ $x = 7$ ” is seen by P_2 before it sees “ $x = 0$ ” and “ $x = 2$ ” although 7 was written to x after the values of 0 and 2.

Strict Hierarchy of the Memory Consistency Models



Synchronization-based Consistency Models:

Weak Consistency

- Consistency conditions apply only to special “**synchronization**” instructions, e.g. barrier synchronization
- Non-sync statements may be executed in any order by various processors.
- E.g. weak consistency, release consistency, entry consistency

Weak consistency:

- All Writes are propagated to other processes, and all Writes done elsewhere are brought locally, at a sync instruction.
 - Accesses to sync variables are sequentially consistent
 - Access to sync variable is not permitted unless all Writes elsewhere have completed
 - No data access is allowed until all previous synchronization variable accesses have been performed
- **Drawback:** cannot tell whether beginning access to shared variables (enter CS), or finished access to shared variables (exit CS).

Contd...

Two types of synchronization Variables: *Acquire* and *Release*

Release Consistency

- *Acquire* indicates CS is to be entered. Hence all *Writes* from other processors should be locally reflected at this instruction
- *Release* indicates access to CS is being completed. Hence, all Updates made locally should be propagated to the replicas at other processors.
- *Acquire* and *Release* can be defined on a subset of the variables.
- If no CS semantics are used, then Acquire and Release act as barrier synchronization variables.
- **Lazy release consistency**: propagate updates on-demand, not the PRAM way.

Entry Consistency

- Each ordinary shared variable is associated with a synchronization variable (e.g., lock, barrier)
- For Acquire /Release on a synchronization variable, access to only those ordinary variables guarded by the synchronization variables is performed.

Shared Memory Mutual Exclusion: Bakery Algorithm (By L. Lamport, 1974)

- **Lamport** proposed the classical bakery algorithm for ***n-process*** mutual exclusion in shared memory systems.
- The algorithm is so called because it mimics the actions that customers follow in a ***bakery store***.
- A process wanting to enter the **critical section (CS)** picks a **token number** that is one greater than the elements in the array choosing $[1..n]$.
- Processes enter the critical section in the increasing order of the token numbers. In case of concurrent accesses to choosing by multiple processes, the processes may have the same token number.
- In this case, a **unique lexicographic order** is defined on the tuple **<token, pid>**, and this dictates the order in which processes enter the critical section.
- The algorithm for **process *i*** is given in Algorithm 13.4.
- The algorithm can be shown to satisfy the three requirements of the critical section problem: (i) mutual exclusion, (ii) bounded waiting, and (iii) progress.

Bakery Algorithm

```
(shared vars)
array of boolean: choosing[1...n];
array of integer: timestamp[1...n];

repeat
(1)  $P_i$  executes the following for the entry section:
(1a)  $\text{choosing}[i] \leftarrow 1$ ;
(1b)  $\text{timestamp}[i] \leftarrow \max_{k \in [1 \dots n]} (\text{timestamp}[k]) + 1$ ;
(1c)  $\text{choosing}[i] \leftarrow 0$ ;
(1d) for count = 1 to n do
(1e)   while  $\text{choosing}[\text{count}]$  do no-op;
(1f)   while  $\text{timestamp}[\text{count}] \neq 0$  and  $(\text{timestamp}[\text{count}], \text{count}) < (\text{timestamp}[i], i)$  do
(1g)     no-op.
(2)  $P_i$  executes the critical section (CS) after the entry section
(3)  $P_i$  executes the following exit section after the CS:
(3a)  $\text{timestamp}[i] \leftarrow 0$ .
(4)  $P_i$  executes the remainder section after the exit section
until false;
```

Algorithm 13.5 Lamport's *n*-process bakery algorithm for shared memory mutual exclusion. Code shown is for process P_i , $1 \leq i \leq n$.

Important Results

- **Mutual exclusion**
 - Role of line (1e)? Wait for others' timestamp choice to stabilize ...
 - Role of line (1f)? Wait for higher priority (lex. lower timestamp) process to enter CS
- **Bounded waiting:** P_i can be overtaken by other processes at most once (each)
- **Progress:** lexicographic order is a total order; process with lowest timestamp in lines (1d)-(1g) enters CS
- **Space complexity:** lower bound of n registers
- **Time complexity:** $O(n)$ time for Bakery algorithm
- **Lamport's fast mutex algorithm takes $O(1)$ time** in the absence of contention. However it compromises on bounded waiting.
 - Uses $W(x) - R(y) - W(y) - R(x)$ sequence necessary and sufficient to check for contention, and safely enter CS

Few Other Algorithms

Author	Year	Algorithm
L. Lamport	1987	Fast mutual exclusion algorithm
G. L. Peterson	1981	The two-process mutual exclusion algorithm
G. L. Peterson and M. Fischer	1977	Modified Mutual exclusion algorithm for two processes
L. Lamport	1977	Concept of Wait-freedom

Conclusion

- **Distributed shared memory (DSM)** is an abstraction whereby distributed programs can communicate with memory operations (***Read and Write***) *as opposed* to using message-passing.
- In this lecture, we have discussed the '**concept of DSM**' and several **consistency models** like linearizability, sequential consistency, causal consistency, pipelined RAM (PRAM), and slow memory.
- Then we have discussed the fundamental problem of '**Shared Memory Mutual Exclusion**' with the help of well-known '**Lamport's Bakery Algorithm**'
- In upcoming lecture, we will discuss about '**Distributed Minimum Spanning Tree**'.

Lecture: 14

Distributed Minimum Spanning Tree



Rajiv Misra

Dept. of Computer Science &
Engineering

Indian Institute of Technology Patna

rajivm@iitp.ac.in

Preface

Recap of Previous Lecture:

- In the previous lecture, we have discussed the basic concepts of '**Distributed Shared Memory**', its consistency models and algorithms for shared memory mutual exclusion.

Content of this Lecture:

- In this lecture, we will discuss about '**Distributed Minimum Spanning Tree**' and
- Also discuss the well known **GHS algorithm** from the following reference:

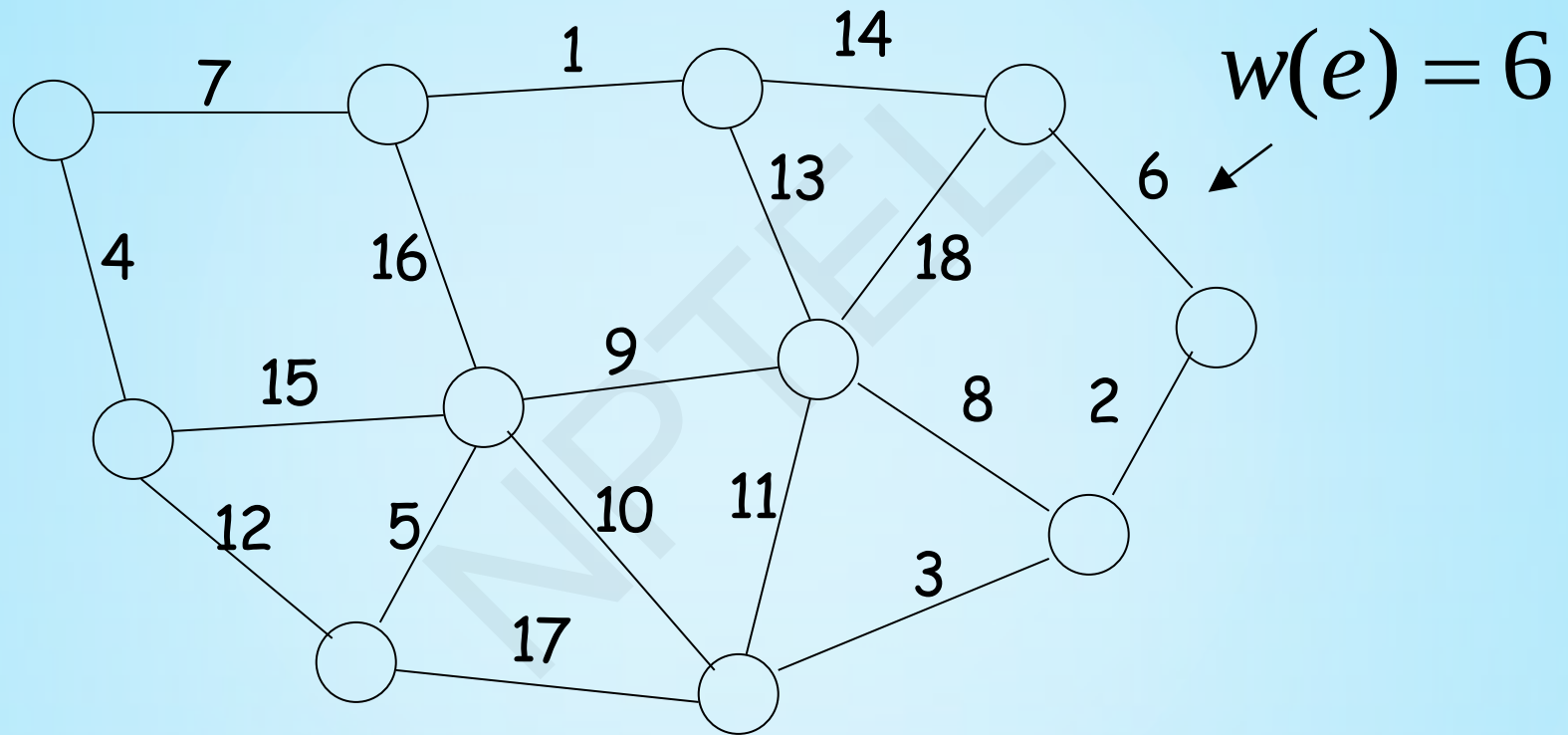
"R.G. Gallager, P.A. Humblet, P.M. Spira, A distributed algorithm for minimum-weight spanning trees, ACM Trans. Program. Lang. Syst. 5 (1983) 66–77."

Introduction

- The **'Distributed Minimum Spanning Tree (MST)'** problem involves the construction of a minimum spanning tree by a distributed algorithm, in a network where nodes communicate by message passing.
- One important application of this problem is to **find a tree that can be used for broadcasting.**
- In particular, if the cost for a message to pass through an edge in a graph is significant, a MST can minimize the total cost for a source process to communicate with all the other processes in the network.

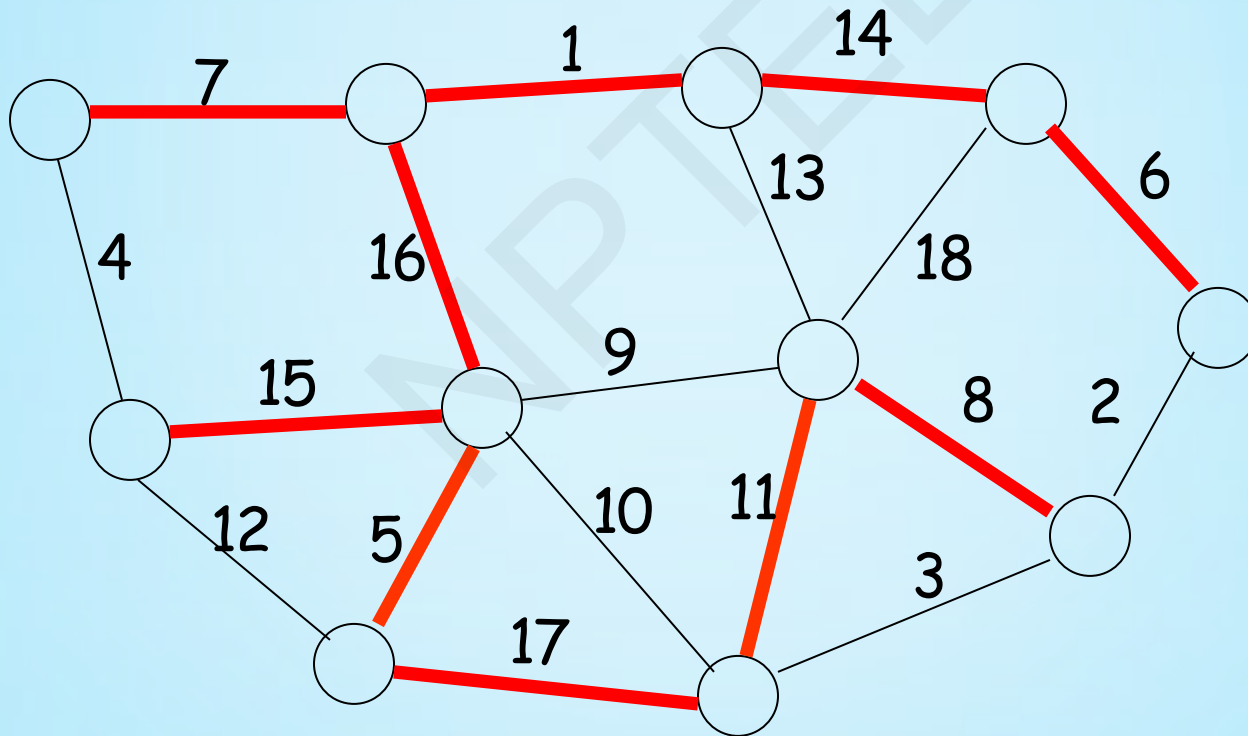
Preliminaries

Weighted Graph $G=(V,E)$, $|V|=n$, $|E|=m$



Spanning tree

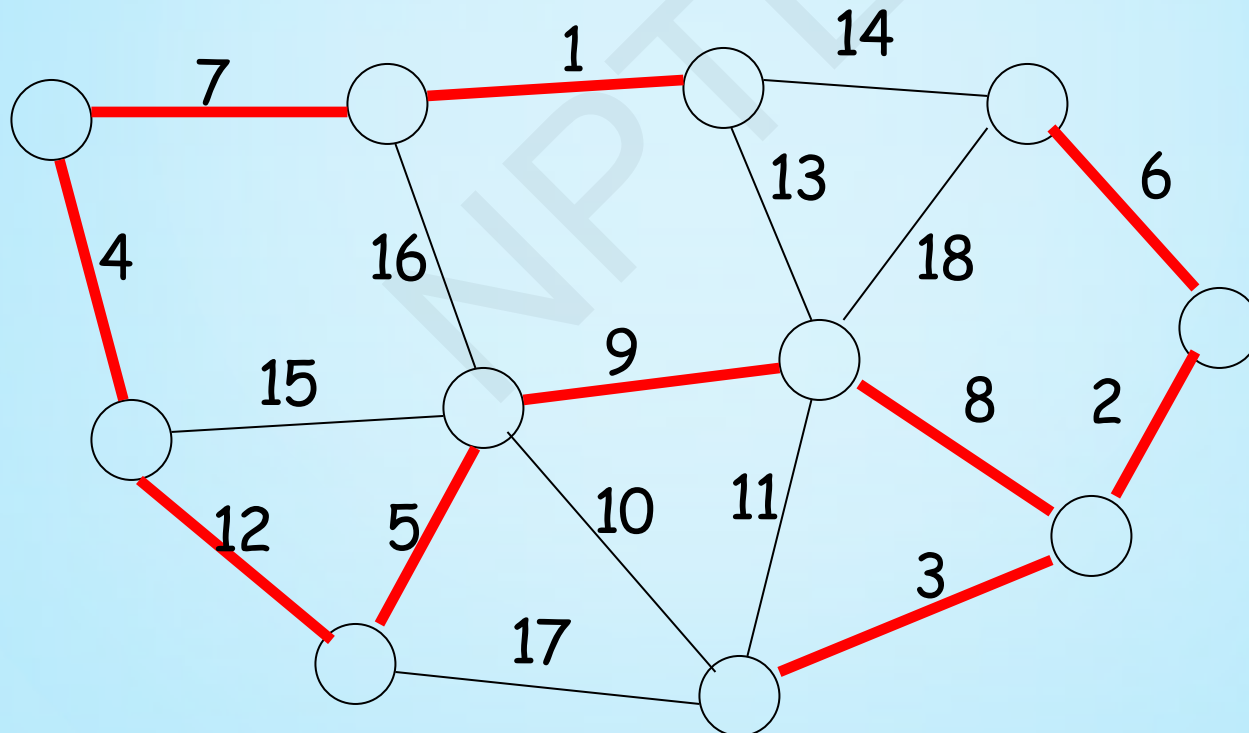
Any tree $T=(V,E')$ (connected acyclic graph)
spanning all the nodes of G



Minimum-weight spanning tree

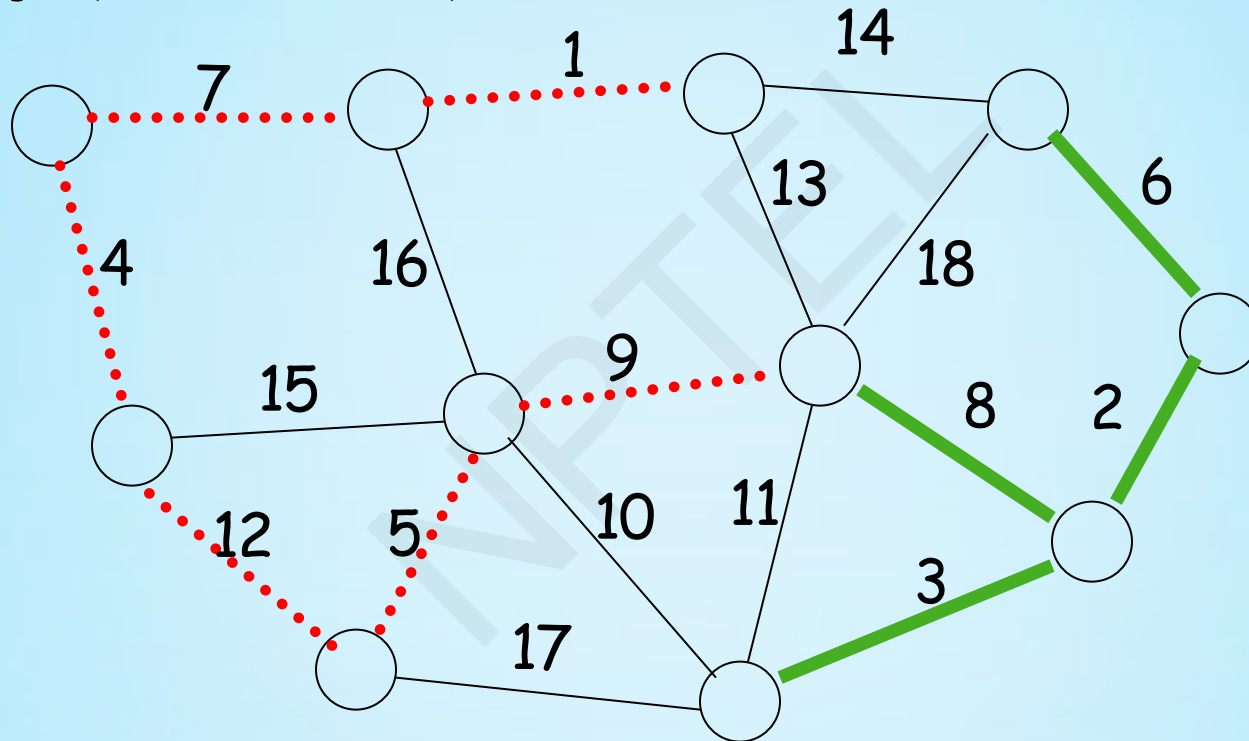
A spanning tree subject to the sum of its weights is minimized:

For MST : $\mathcal{T}$ $w(\mathcal{T}) = \sum_{e \in \mathcal{T}} w(e)$ is minimized



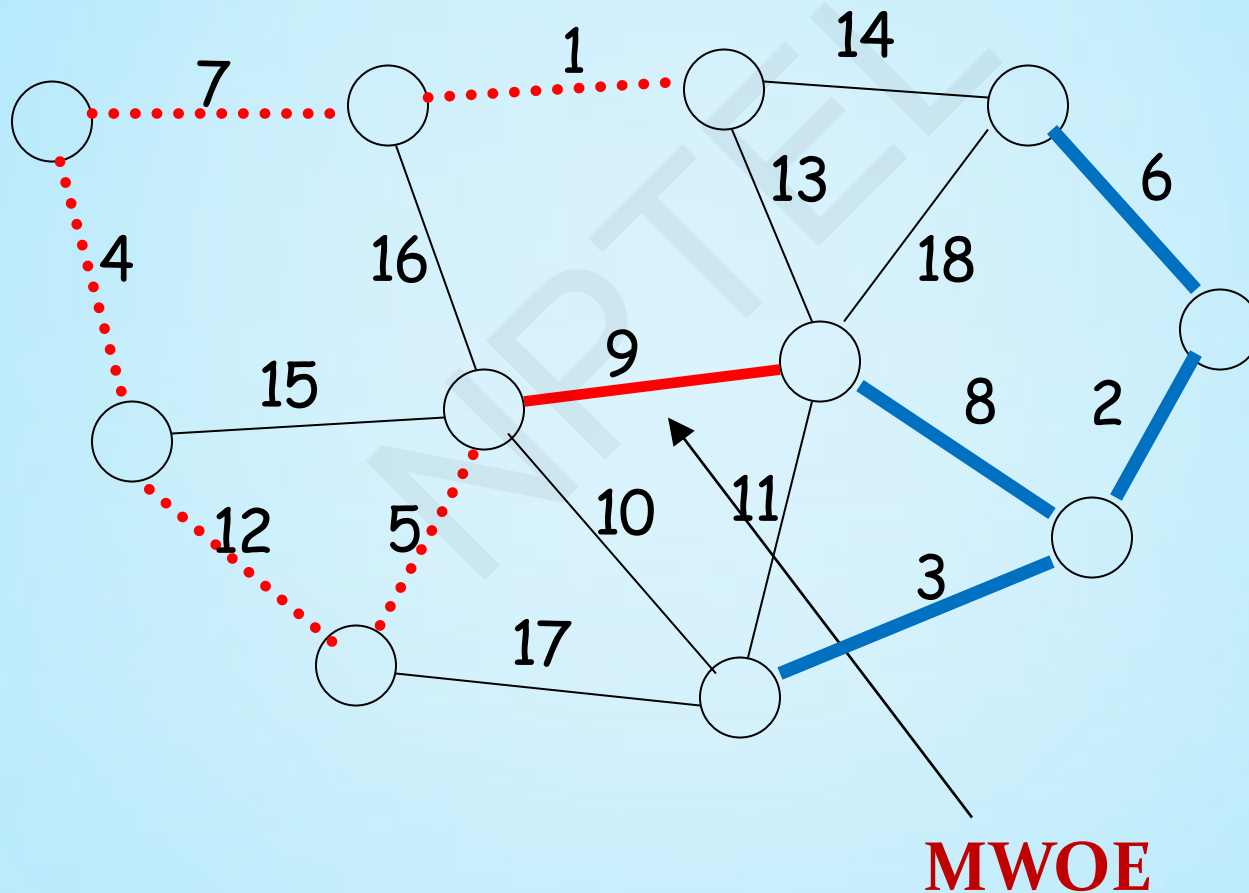
Spanning tree fragment

Any (connected) sub-tree of a MST



Minimum weight outgoing edge (MWOE)

An edge adjacent to the fragment with smallest weight and that does not create a cycle

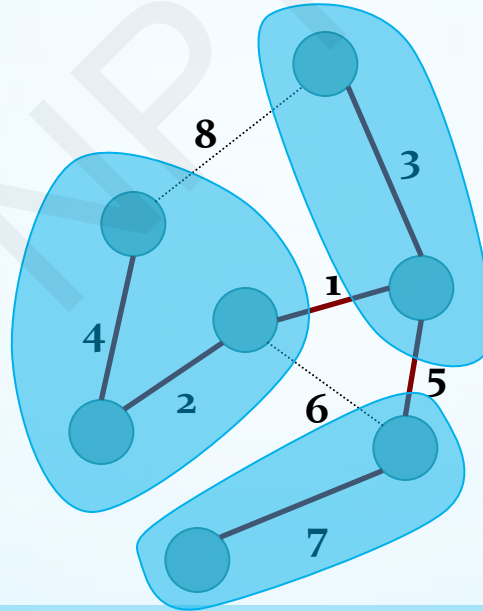


MST Fragment

- Definition – MST *fragment*

- A connected sub-tree of MST

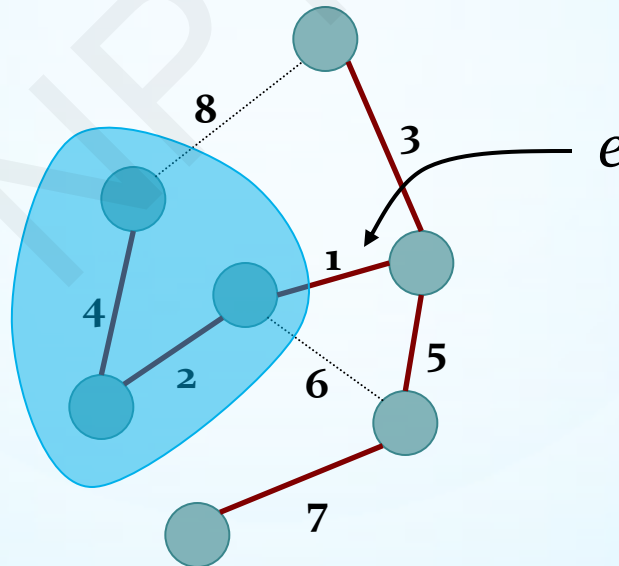
Example of possible fragments:



MST Properties

- MST Property 1

Given a fragment of an MST, let e be a minimum-weight outgoing edge of the fragment. Then joining e and its adjacent non-fragment node to the fragment yields another fragment of an MST.

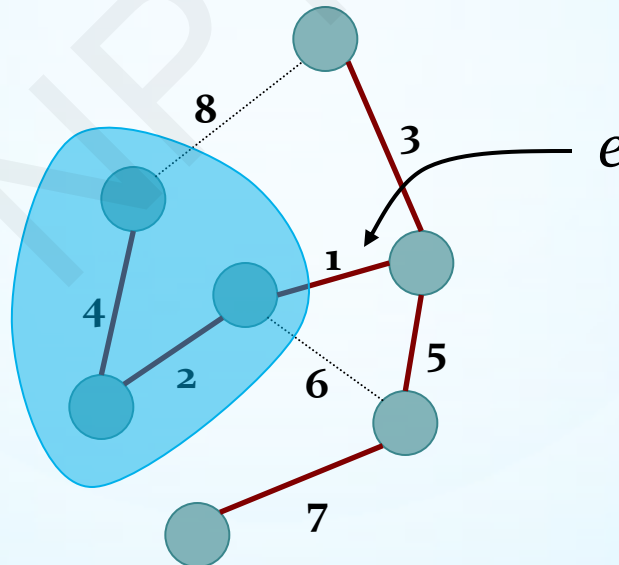


MST Properties

- MST Property 1

Given a fragment of an MST, let e be a minimum-weight outgoing edge of the fragment. Then joining e and its adjacent non-fragment node to the fragment yields another fragment of an MST.

- *Proof:*



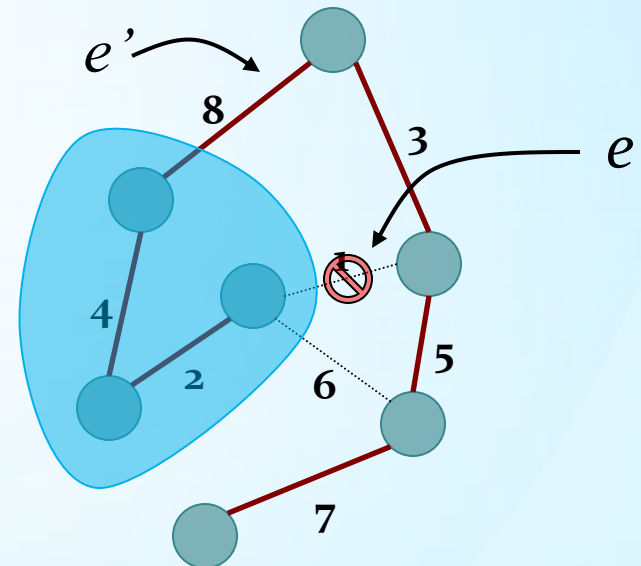
MST Properties

- MST Property 1

Given a fragment of an MST, let e be a minimum-weight outgoing edge of the fragment. Then joining e and its adjacent non-fragment node to the fragment yields another fragment of an MST.

- *Proof:*

Suppose e is not in MST, but some e' instead.



MST Properties

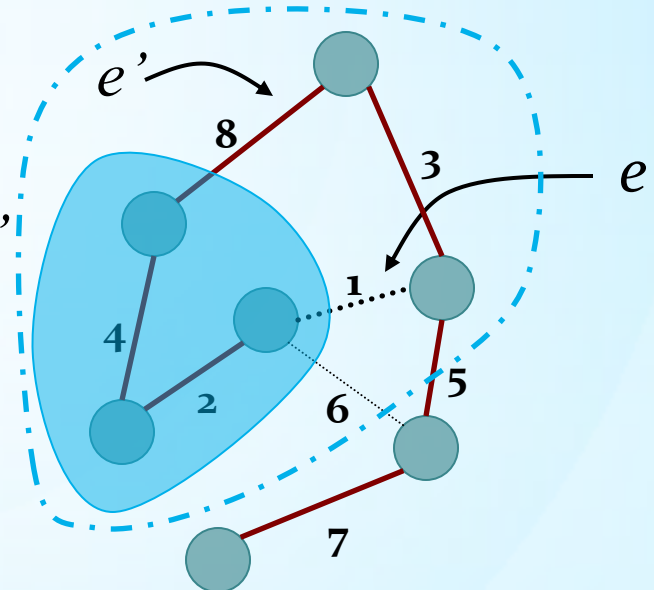
- MST Property 1

Given a fragment of an MST, let e be a minimum-weight outgoing edge of the fragment. Then joining e and its adjacent non-fragment node to the fragment yields another fragment of an MST.

- *Proof:*

Suppose e is not in MST, but some e' instead.

MST with e forms a cycle.



MST Properties

- MST Property 1

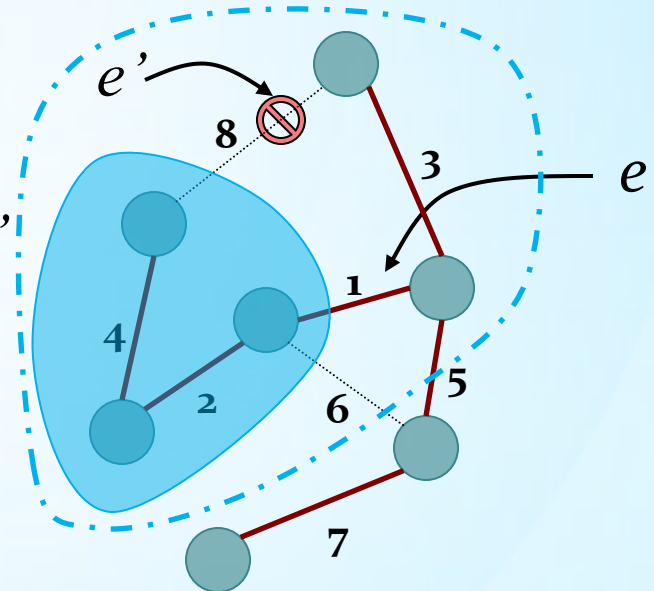
Given a fragment of an MST, let e be a minimum-weight outgoing edge of the fragment. Then joining e and its adjacent non-fragment node to the fragment yields another fragment of an MST.

- *Proof:*

Suppose e is not in MST, but some e' instead.

MST with e forms a cycle.

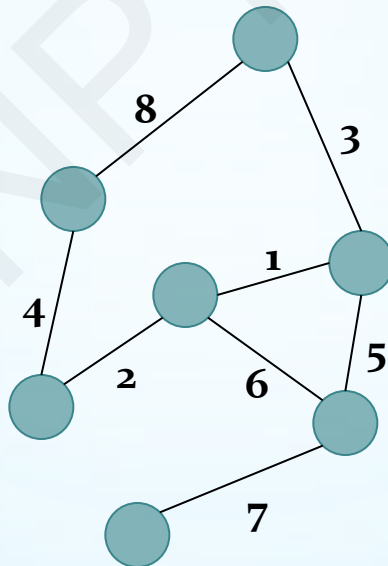
We obtain a cheaper MST with e instead of e' .



MST Properties

- MST Property 2

If all the edges of a connected graph have different weights, then the MST is unique.

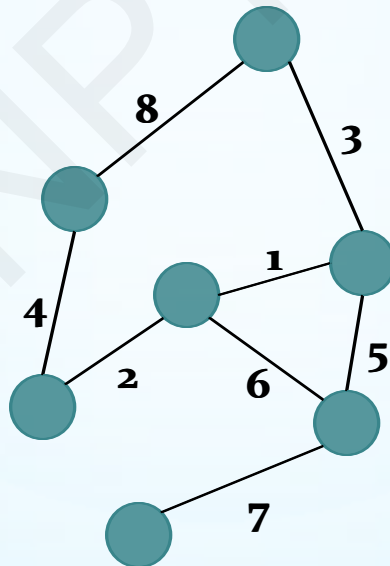


MST Properties

- MST Property 2

If all the edges of a connected graph have different weights, then the MST is unique.

- *Proof:*



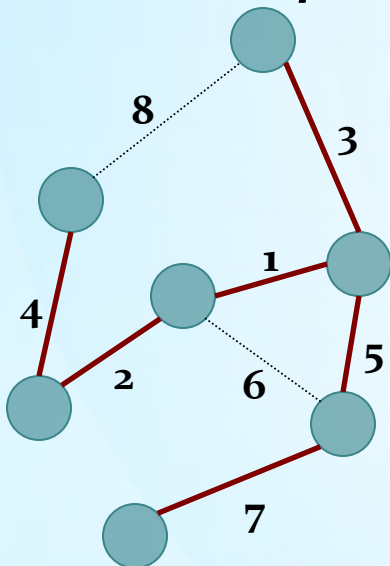
MST Properties

- MST Property 2

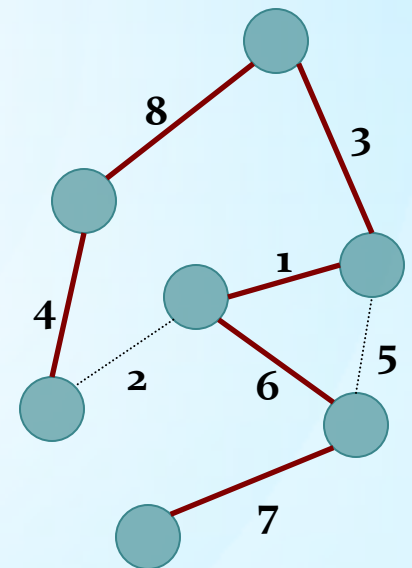
If all the edges of a connected graph have different weights, then the MST is unique.

- *Proof:*

Suppose existence of two MSTs.



T



T'

MST Properties

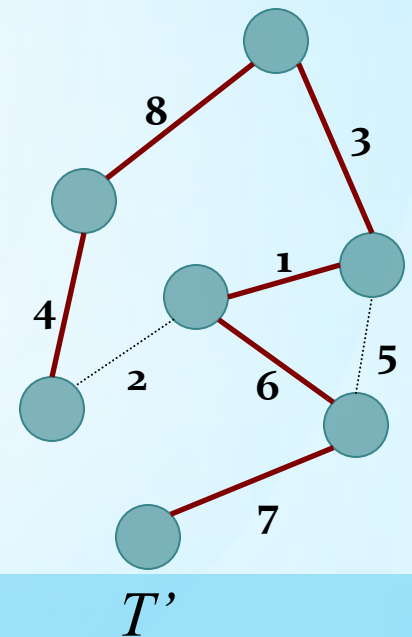
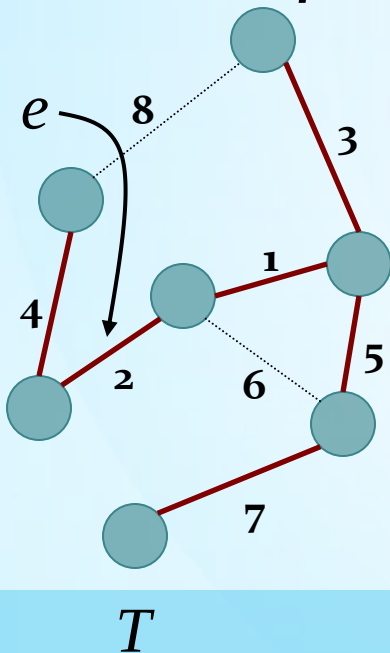
- MST Property 2

If all the edges of a connected graph have different weights, then the MST is unique.

- *Proof:*

Suppose existence of two MSTs.

Let e be the minimal-weight edge not in both MSTs (wlog e in T).

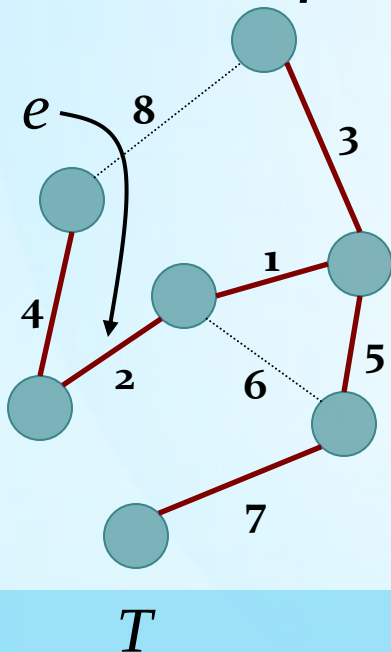


MST Properties

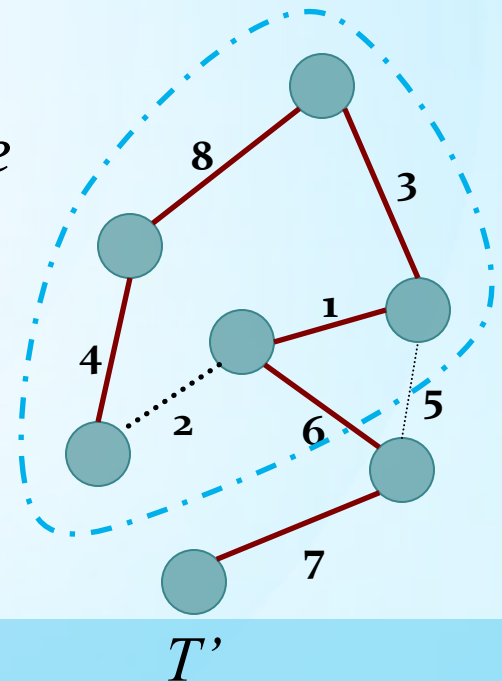
- MST Property 2

If all the edges of a connected graph have different weights, then the MST is unique.

- *Proof:*



*Suppose existence of two MSTs.
Let e be the minimal-weight edge not in both MSTs (wlog e in T).
 T' with e has a cycle*

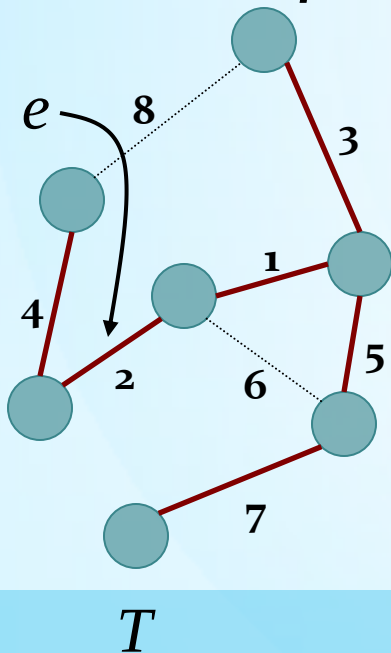


MST Properties

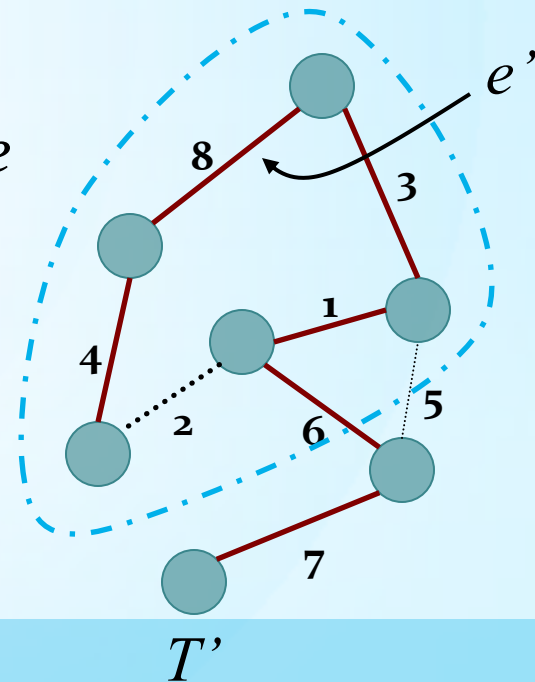
- MST Property 2

If all the edges of a connected graph have different weights, then the MST is unique.

- *Proof:*



*Suppose existence of two MSTs.
Let e be the minimal-weight edge not in both MSTs (wlog e in T).
 T' with e has a cycle
At least cycle edge e' is not in T .*

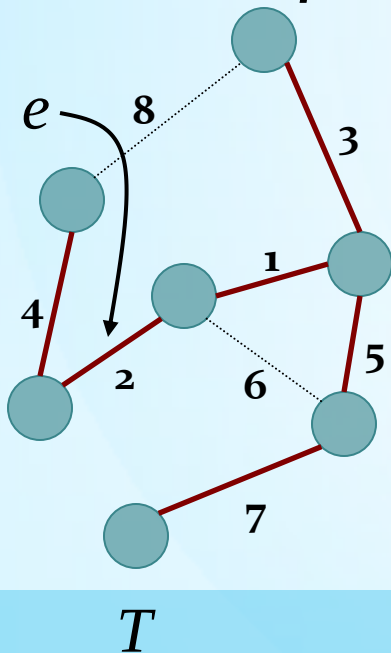


MST Properties

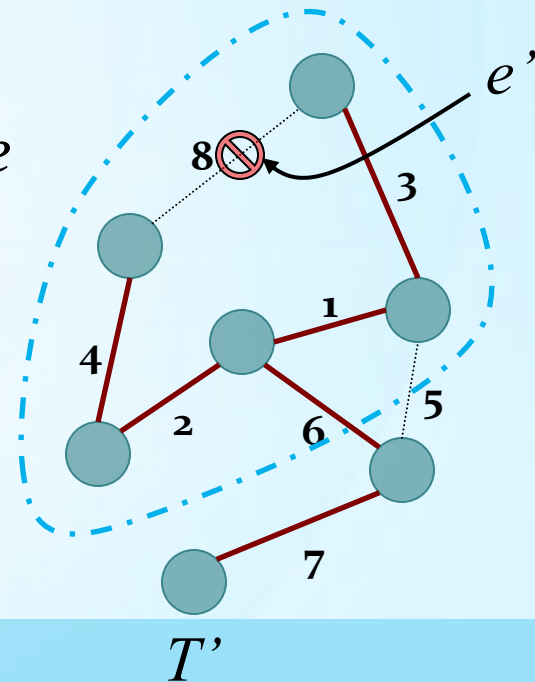
- MST Property 2

If all the edges of a connected graph have different weights, then the MST is unique.

- Proof:

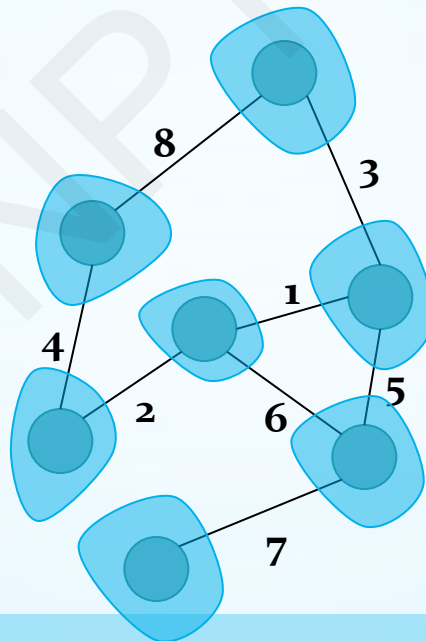


Suppose existence of two MSTs.
Let e be the minimal-weight edge not in both MSTs (wlog e in T).
 T' with e has a cycle
At least cycle edge e' is not in T .
Since $w(e) < w(e')$ we conclude that T' with e and without e' is a smaller MST than T' .



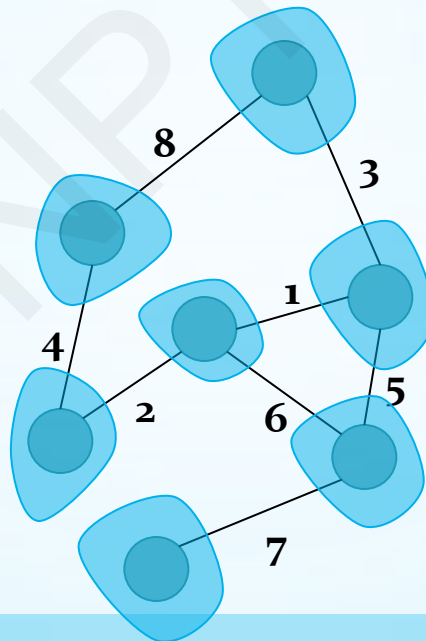
MST Properties

- Idea of MST based on properties 1 & 2
 - Start with fragments of one node.



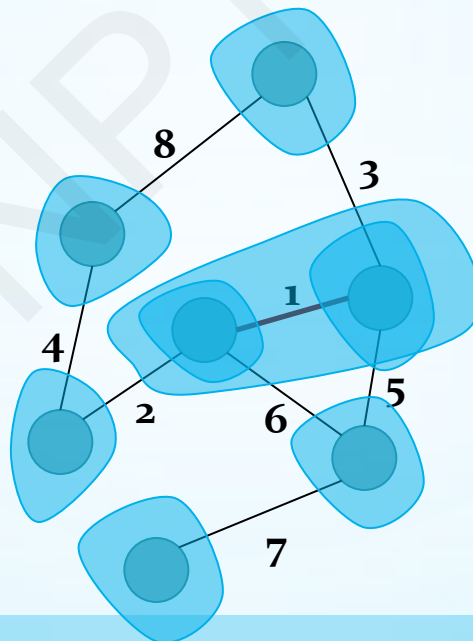
MST Properties

- Idea of MST based on properties 1 & 2
 - Enlarge fragments in any order (property 1)
 - Combine fragments with a common node (property 2)
 - (property 2)



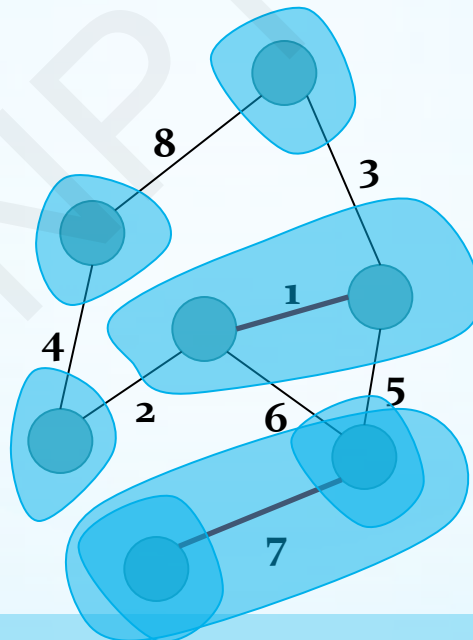
MST Properties

- Idea of MST based on properties 1 & 2
 - Enlarge fragments in any order (property 1)
 - Combine fragments with a common node
 - (property 2)



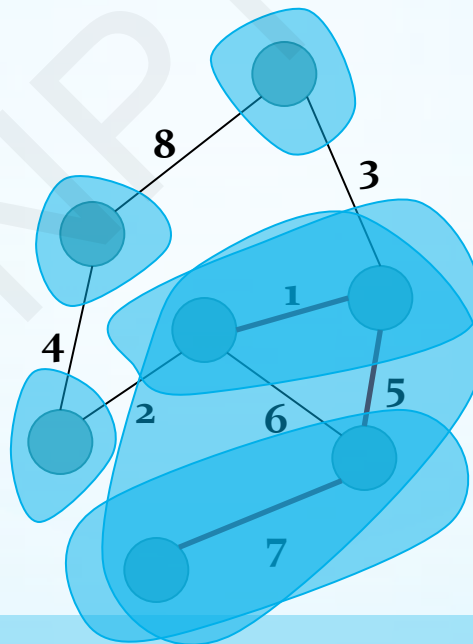
MST Properties

- Idea of MST based on properties 1 & 2
 - Enlarge fragments in any order (property 1)
 - Combine fragments with a common node
 - (property 2)



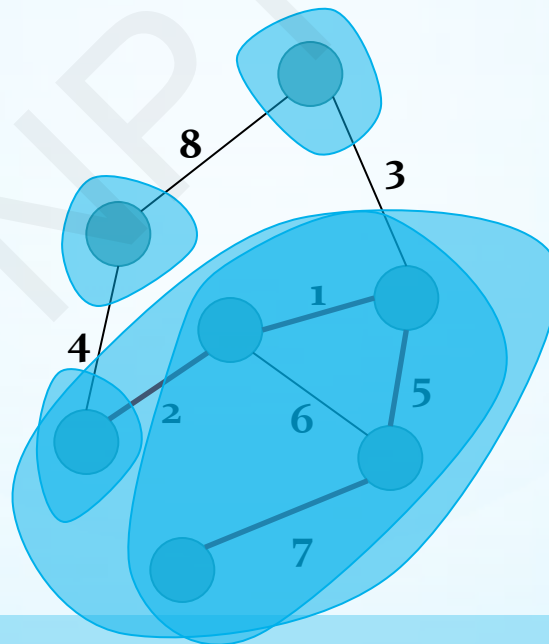
MST Properties

- Idea of MST based on properties 1 & 2
 - Enlarge fragments in any order (property 1)
 - Combine fragments with a common node
 - (property 2)



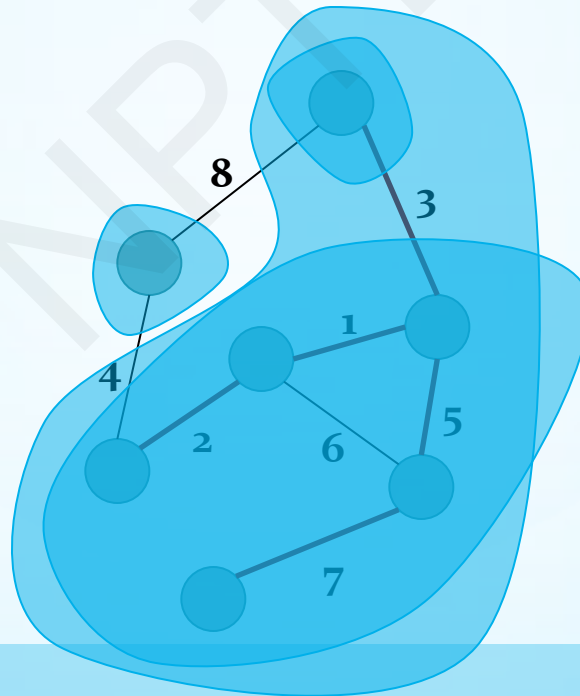
MST Properties

- Idea of MST based on properties 1 & 2
 - Enlarge fragments in any order (property 1)
 - Combine fragments with a common node
 - (property 2)



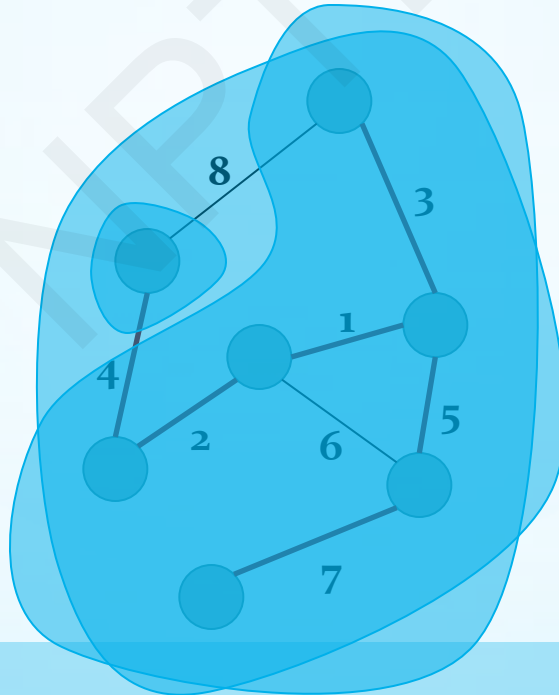
MST Properties

- Idea of MST based on properties 1 & 2
 - Enlarge fragments in any order (property 1)
 - Combine fragments with a common node
 - (property 2)



MST Properties

- Idea of MST based on properties 1 & 2
 - Enlarge fragments in any order (property 1)
 - Combine fragments with a common node
 - (property 2)



MST in Message Passing Model

- The **message-passing model** is one of the most commonly used models in distributed computing. In this model, **each process is modeled as a node** of a graph. The **communication channel between two processes is an edge** of the graph.
- Two commonly used algorithms for the **classical minimum spanning tree problem** are **Prim's algorithm and kruskal's algorithm**. However, it is difficult to apply these two algorithms in the distributed message-passing model. The **main challenges** are:
 - i. Both Prim's algorithm and Kruskal's algorithm require **processing one node or vertex at a time**, making it difficult to make them run in parallel.
 - ii. Both Prim's algorithm and Kruskal's algorithm require **processes to know the state of the whole graph**, which is very difficult to discover in the message-passing model.
- Due to these difficulties, new techniques were needed for **distributed MST algorithms** in the message-passing model.

GHS Algorithm

GHS Algorithm

- The **GHS algorithm of 'Gallager, Humblet and Spira' (1983)** is one of the best-known algorithms in distributed computing theory.
- GHS is a distributed algorithm based on Kruskal's algorithm** that constructs the minimum-weight spanning tree in a **connected undirected graph with distinct edge weights**.
- A processor exists at each node of the graph, knowing initially only the weights of the adjacent edges. The processors obey the same algorithm and exchange messages with neighbors until the tree is constructed.
- This algorithm can construct the **minimum spanning tree in asynchronous Message-passing model**.

GHS Algorithm: Synchronous vs. Asynchronous

1) Synchronous GHS:

- Works with non-uniform models, distinct weights

Steps:

1. Initially, each node is a fragment
2. **Repeat in parallel:** (Synchronous Phase)
 - i. Each fragment – coordinated by a fragment root node - finds its Minimum Weight Outgoing Edge (MWOE)
 - ii. Merge fragments adjacent to MWOE's
3. Until there is only one fragment left

2) Asynchronous GHS:

- Simulates the synchronous version
- Works with uniform models, distinct weights

Steps:

1. Every fragment F has a level $L(F) \geq 0$: at the beginning, each node is a fragment of level 0
2. Two types of merges: absorption and join

Note: We will work with the 'Asynchronous GHS Algorithm'

Overview

- The input graph $G(V,E)$ is considered to be a network, where **vertices V** are independent computing nodes and **edges E** are communication links. Links are weighted as in the classical problem.
- At the beginning of the algorithm, **nodes know only the weights of the links which are connected to them.** (It is possible to consider models in which they know more, for example their neighbors' links.)
- As the output of the algorithm, **every node knows which of its links belong to the Minimum Spanning Tree and which do not.**

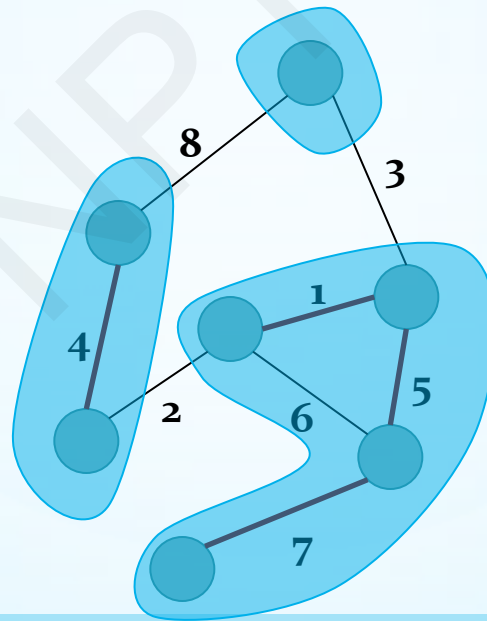
Preconditions

1. The algorithm should run on a **connected undirected graph**.
2. The graph should have **distinct finite weights** assigned to each edge.
3. Each node initially knows the weight for each edge incident to that node.
4. Initially, each node is in a quiescent state and it either spontaneously awakens or is awakened by receipt of any message from another node.
5. Messages can be transmitted independently in both directions on an edge and arrive after an unpredictable but finite delay, without error.
6. Each edge delivers messages in **FIFO order**.

The Idea of Distributed MST

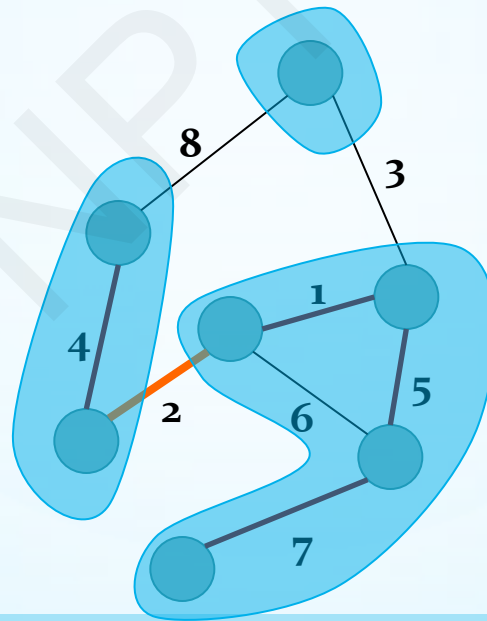
GHS Notation: Fragments

- Fragments
 - Every node starts as a single fragment.



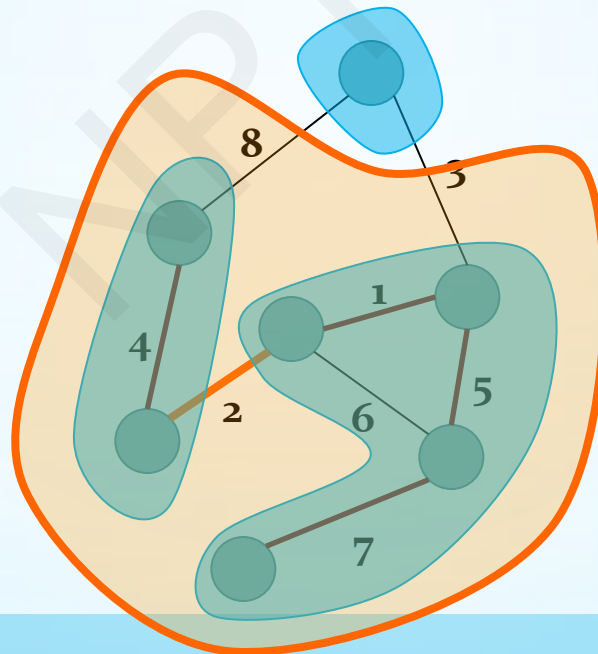
GHS Notation: Fragments

- Fragments
 - Each fragment finds its minimum outgoing edge.



GHS Notation: Fragments

- Fragments
 - Each fragment finds its minimum outgoing edge.
 - Then it tries to combine with the adjacent fragment.

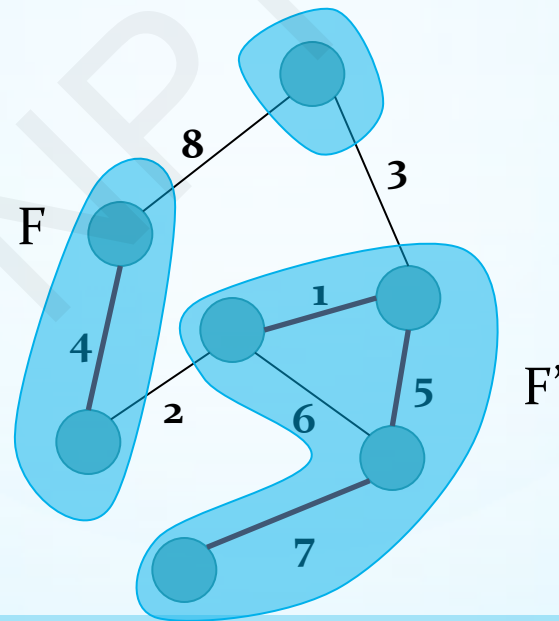


GHS Notation: Levels

- Levels
 - Every fragment has an associated level that
 - has impact on combining fragments.
 - A fragment with a single node is defined to
 - to be at level 0.

GHS Notation: Levels

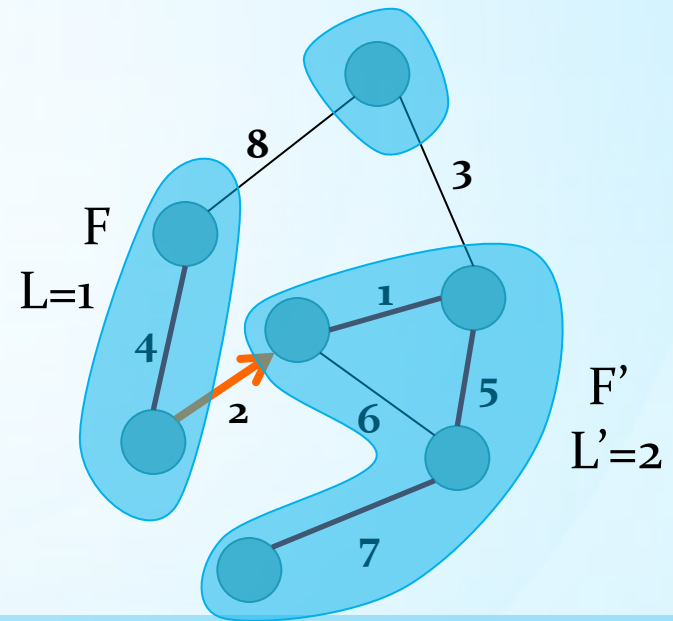
- Levels
 - The combination of two fragments depends on
 - the levels of fragments.



GHS Notation: Levels

- Levels
 - The combination of two fragments depends on
 - the levels of fragments.

If a fragment F wishes to connect to a fragment F' and $L < L'$ then:

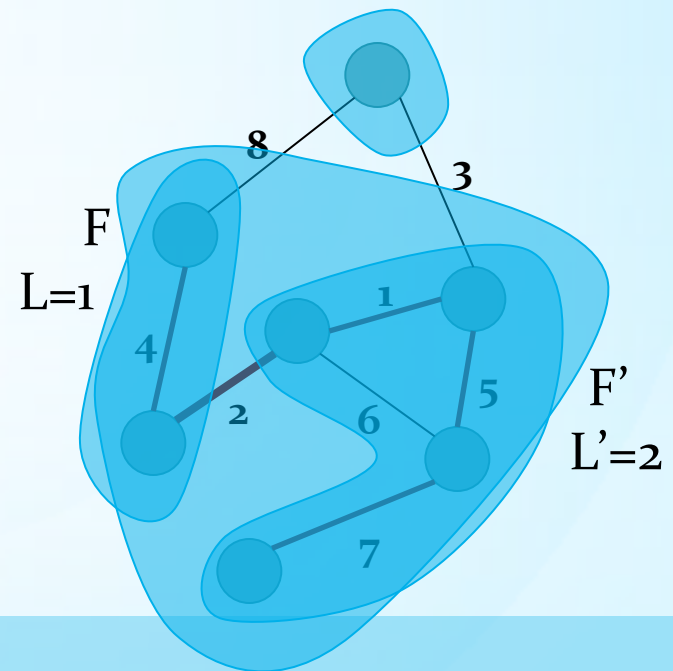


GHS Notation: Levels

- Levels
 - The combination of two fragments depends on
 - the levels of fragments.

If a fragment F wishes to connect to a fragment F' and $L < L'$ then:

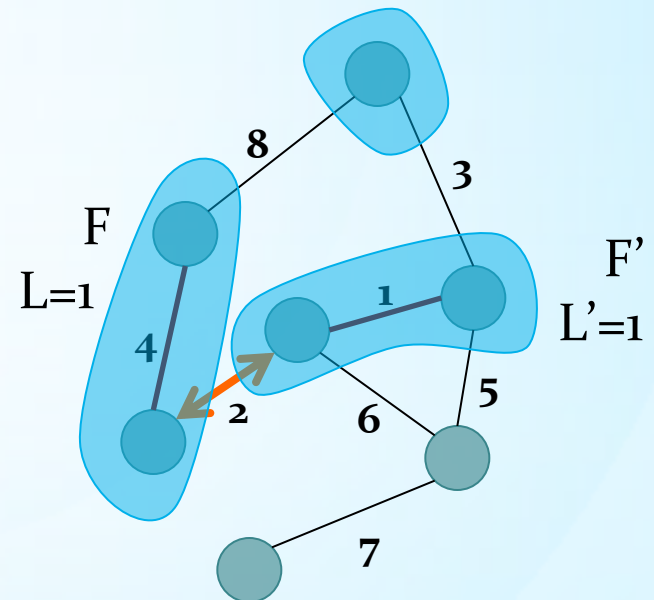
F is absorbed in F' and the resulting fragment is at level L' .



GHS Notation: Levels

- Levels
 - The combination of two fragments depends on
 - the levels of fragments.

If fragments F and F' have the same minimum outgoing edge and $L = L'$ then:

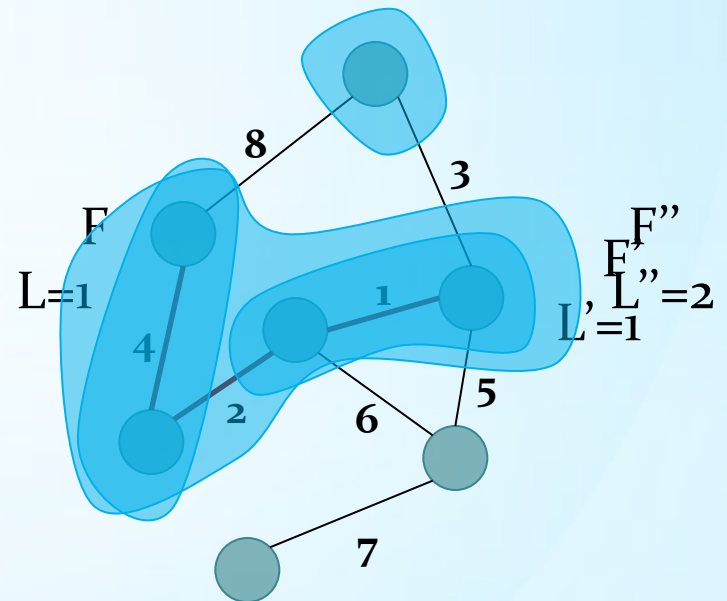


GHS Notation: Levels

- Levels
 - The combination of two fragments depends on
 - the levels of fragments.

If fragments F and F' have the same minimum outgoing edge and $L = L'$ then:

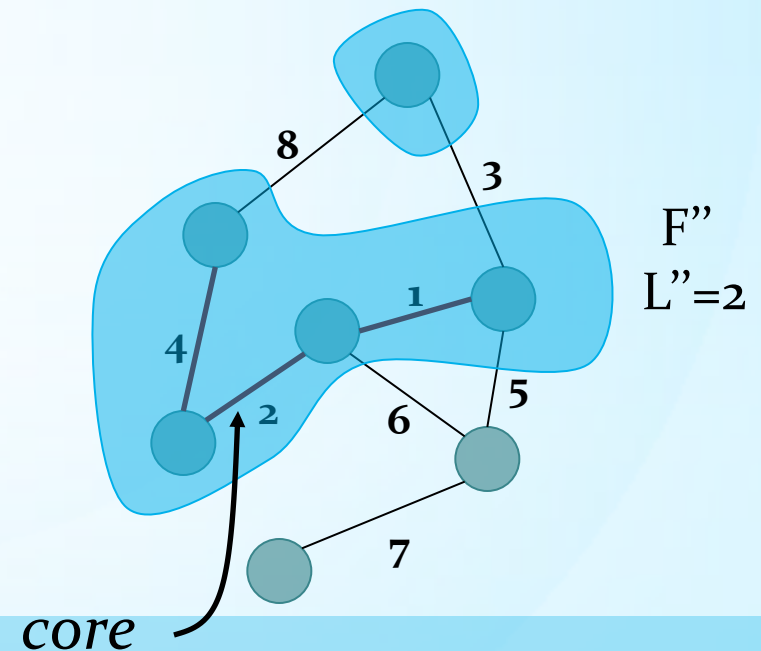
The fragments combine into a new fragment F'' at level $L'' = L + 1$.



GHS Notation: Levels

- Levels
 - The identity of a fragment is the weight of its
 - *core*.

If fragments F and F' with same level were combined, the combining edge is called the core of the new segment.



GHS Notation: Node States

- State
 - Each node has a state

Sleeping - initial state

Find - during fragment's search
for a minimal outgoing edge

Found - otherwise (when a
minimal outgoing edge was
found)

The Algorithm

Description of the Algorithm (1)

- The GHS algorithm assigns a level to each fragment, which is a non decreasing integer with initial value 0.
- Each non-zero level fragment has an ID, which is the ID of the core edge in the fragment, which is selected when the fragment is constructed.
- During the execution of the algorithm, each node can classify each of its incident edges into three categories:
 - (i) **Branch edges** are those that have already been determined to be part of the MST.
 - (ii) **Rejected edges** are those that have already been determined not to be part of the MST.
 - (iii) **Basic edges** are neither branch edges nor rejected edges.

Description of the Algorithm (2)

- **For level-0 fragments**, each **awakened node** will do the following:
 1. Choose its minimum-weight incident edge and **marks that edge as a branch edge**.
 2. Send a **message via the branch edge** to notify the node on the other side.
 3. **Wait for a message** from the other end of the edge.
- The edge chosen by both nodes it **connects becomes the core** with level 1.

Description of the Algorithm (3)

- **For a non-zero level fragment**, an execution of the algorithm can be separated into three stages in each level:

(i) Broadcast

- The two nodes adjacent to the core **broadcast messages to the rest of the nodes in the fragment**. The messages are **sent via the branch edge** but not via the core.
- Each broadcast message contains the ID and level of the fragment. At the end of this stage, each node has received the new fragment ID and level.

Description of the Algorithm (4)

(ii) Convergecast

- In this stage, all nodes in the fragment cooperate to **find the minimum weight outgoing edge of the fragment**. Outgoing edges are edges connecting to other fragments.
- The messages sent in this stage are in the **opposite direction of the broadcast stage**. Initialized by all the leaves (the nodes that have only one branch edge), a message is sent through the branch edge.
- The message contains the minimum weight of the incident outgoing edge it found (or infinity if no such edge was found). For each non-leaf node, (let the number of its branch edges be n) after receiving $n-1$ convergecast messages, it will pick the minimum weight from the messages and compare it to the weights of its incident outgoing edges. The smallest weight will be sent toward the branch it received the broadcast from.

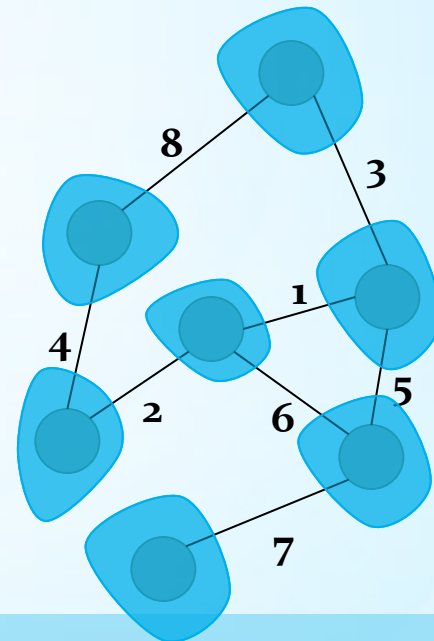
Description of the Algorithm (5)

(iii) Change core

- After the completion of the previous stage, the two nodes connected by the core can **inform each other of the best edges they received.**
- Then they can identify the **minimum outgoing edge from the entire fragment.** A message will be sent from the core to the minimum outgoing edge via a path of branch edges.
- Finally, a message will be sent out via the chosen outgoing edge to request to combine the two fragments that the edge connects.

The Algorithm: Execution

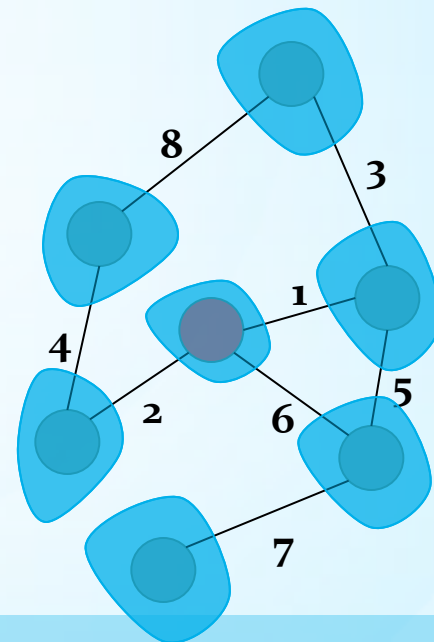
- Fragment minimum outgoing edge discovery
 - Special case of zero-level fragment (*Sleeping*).



The Algorithm: Execution

- Fragment minimum outgoing edge discovery
 - Special case of zero-level fragment (***Sleeping***).

*When a node awakes from the state ***Sleeping***, it finds a minimum edge connected.*

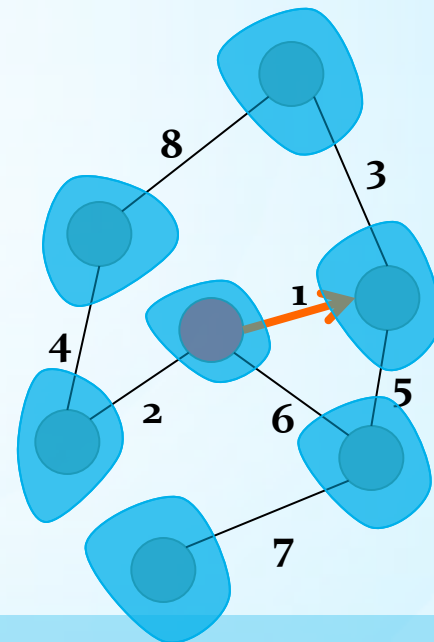


The Algorithm: Execution

- Minimum outgoing edge discovery
 - Special case of zero-level fragment (***Sleeping***).

*When a node awakes from the state ***Sleeping***, it finds a minimum edge connected.*

*Marks it as a **branch** of MST and sends a **Connect** message over this edge.*



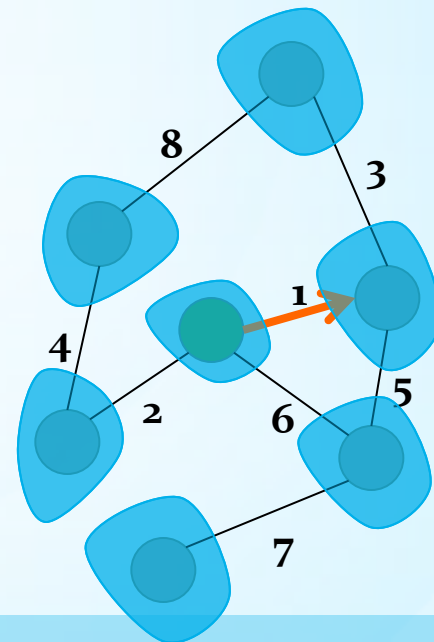
The Algorithm: Execution

- Minimum outgoing edge discovery
 - Special case of zero-level fragment (***Sleeping***).

*When a node awakes from the state ***Sleeping***, it finds a minimum edge connected.*

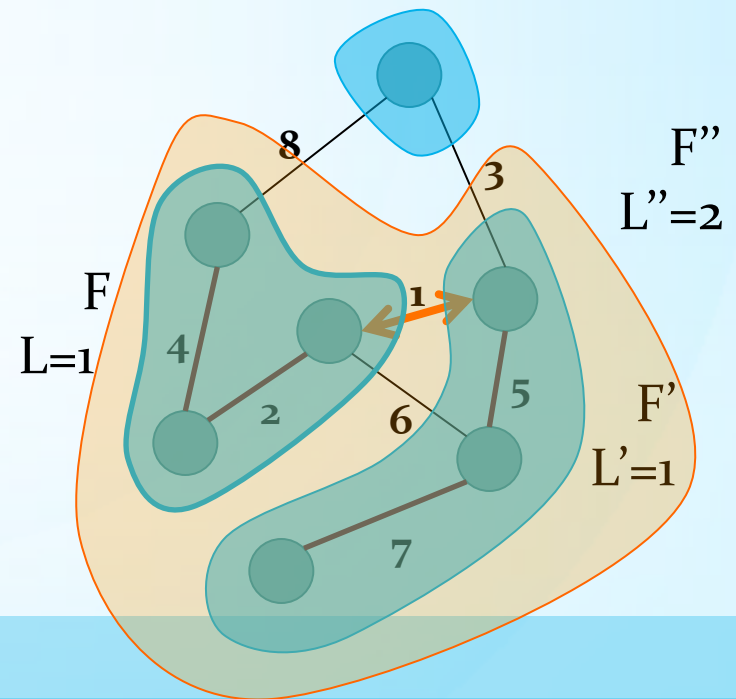
*Marks it as a **branch** of MST and sends a **Connect** message over this edge.*

*Goes into a **Found** state.*



The Algorithm: Execution

- Minimum outgoing edge discovery
 - Take a fragment at level L that was just combined
 - out of two level $L-1$ fragments.

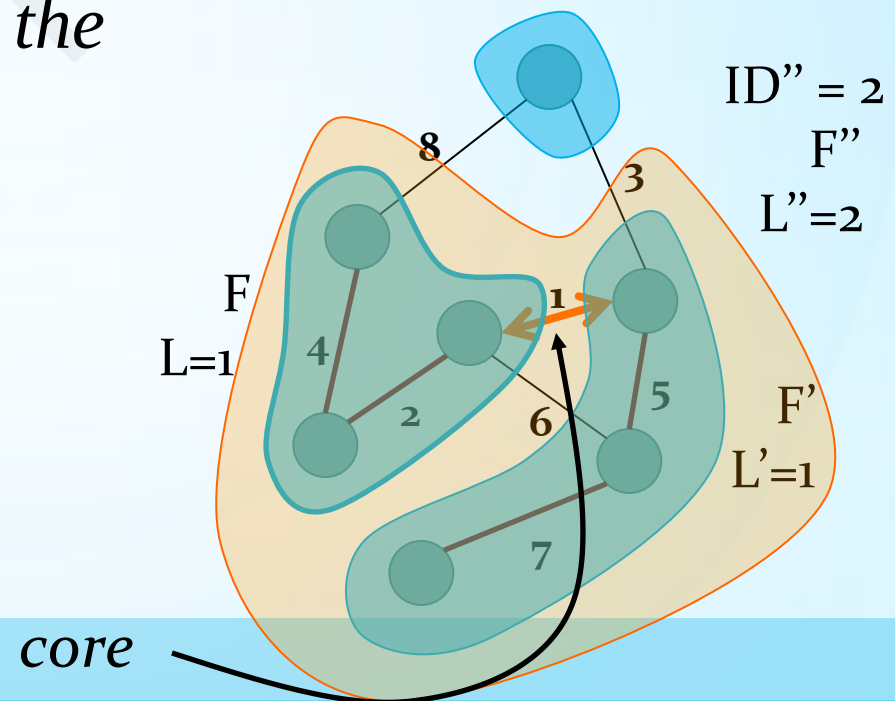


The Algorithm: Execution

- Minimum outgoing edge discovery
 - Take a fragment at level L that was just combined
 - out of two level L-1 fragments.

The weight of the core is the identity of the fragment.

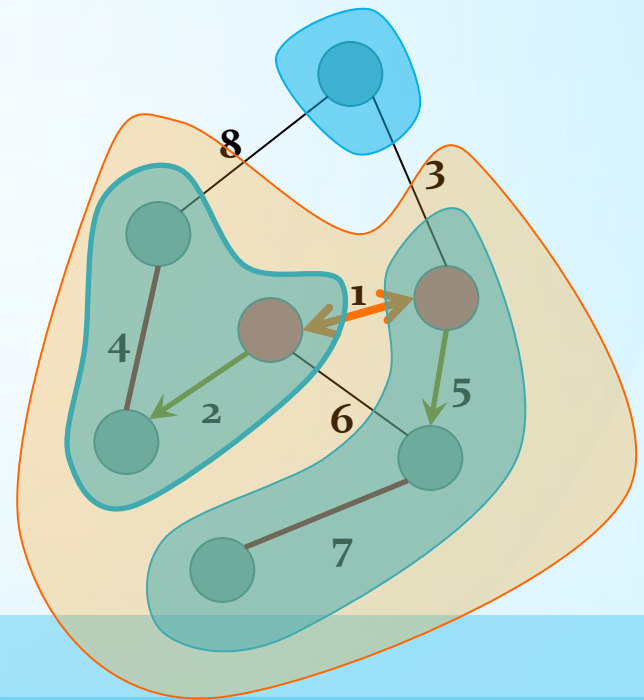
It acts as a root of a fragment tree.



The Algorithm: Execution

- Minimum outgoing edge discovery
 - Take a fragment at level L that was just combined
 - out of two level L-1 fragments.

*Nodes adjacent to core send an **Initiate** message to the borders.*

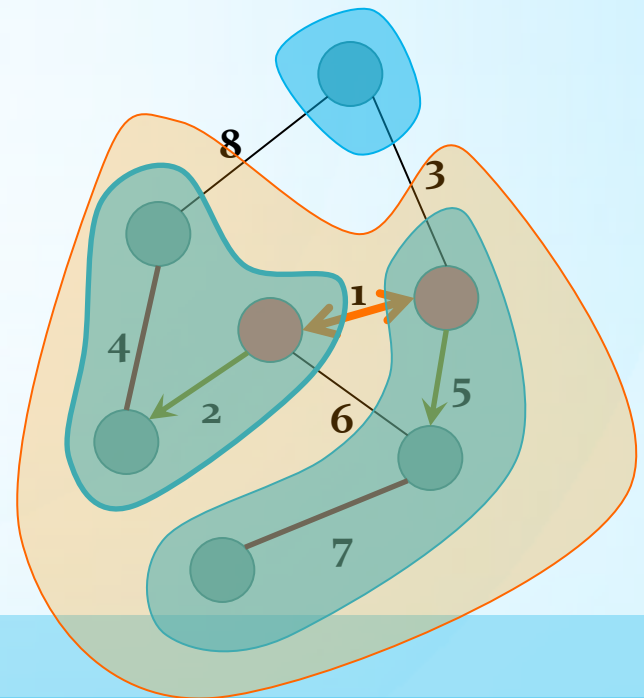


The Algorithm: Execution

- Minimum outgoing edge discovery
 - Take a fragment at level L that was just combined
 - out of two level L-1 fragments.

*Nodes adjacent to core send an **Initiate** message to the borders.*

Relayed by the intermediate nodes in the fragment.



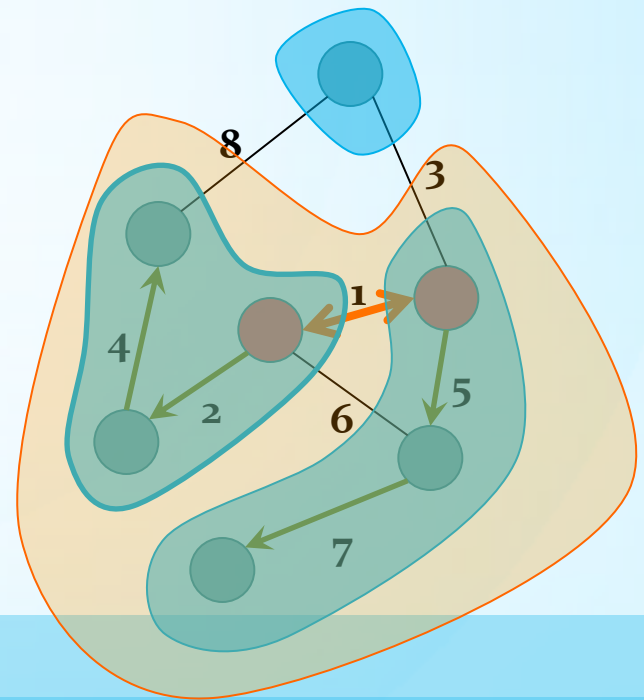
The Algorithm: Execution

- Minimum outgoing edge discovery
 - Take a fragment at level L that was just combined
 - out of two level L-1 fragments.

*Nodes adjacent to core send an **Initiate** message to the borders.*

Relayed by the intermediate nodes in the fragment.

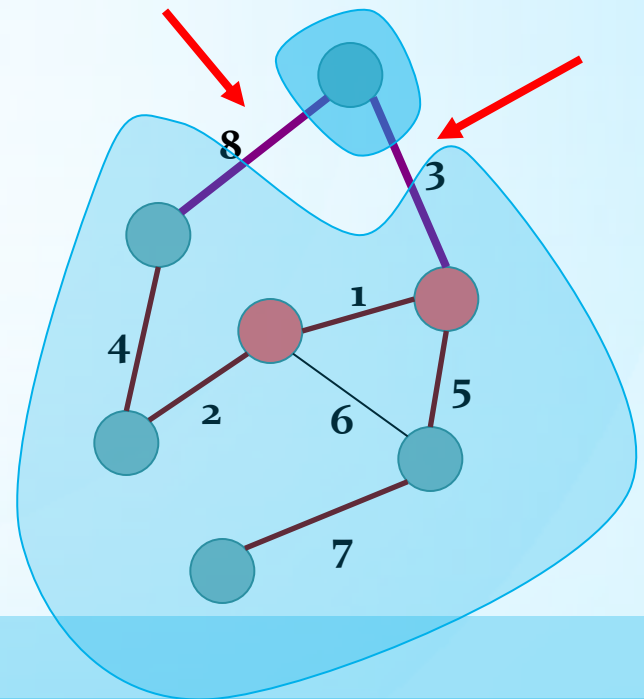
*Puts the nodes in the **Find** state.*



The Algorithm: Execution

- Minimum outgoing edge discovery
 - Edge classification/

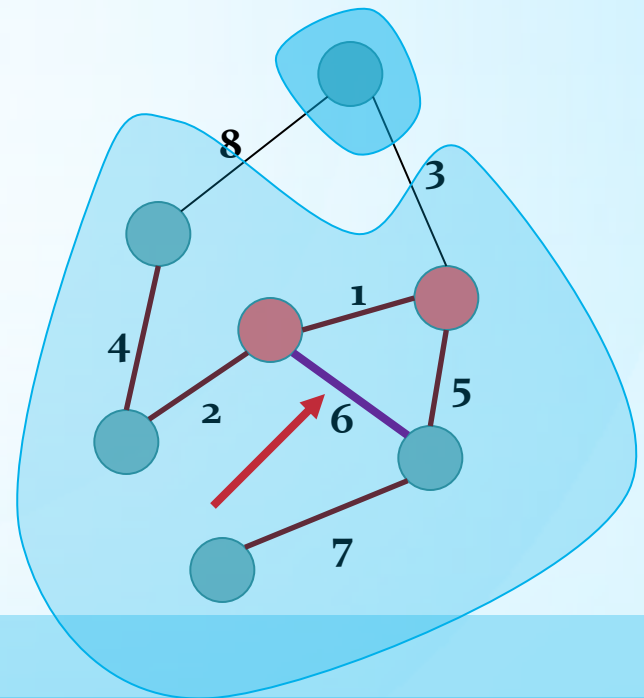
Basic - yet to be classified, can be inside fragment or outgoing edges



The Algorithm: Execution

- Minimum outgoing edge discovery
 - Edge classification.

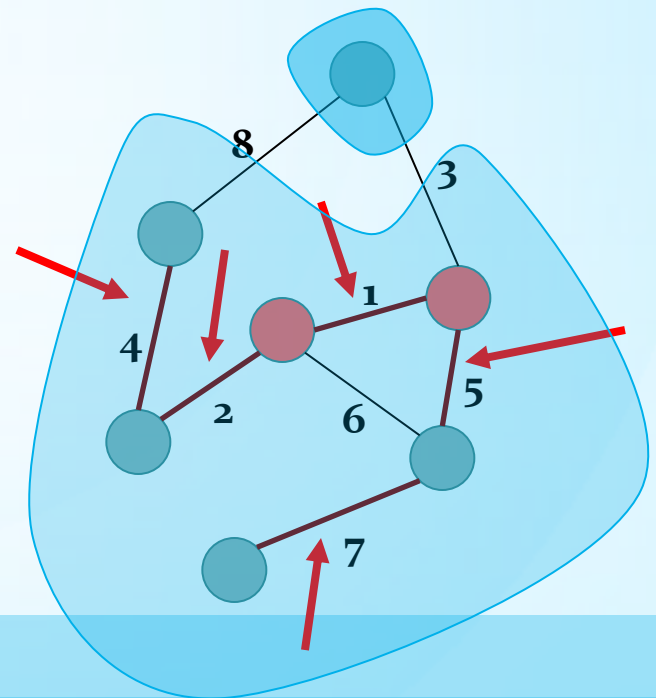
Rejected – an inside fragment edge



The Algorithm: Execution

- Minimum outgoing edge discovery
 - Edge classification.

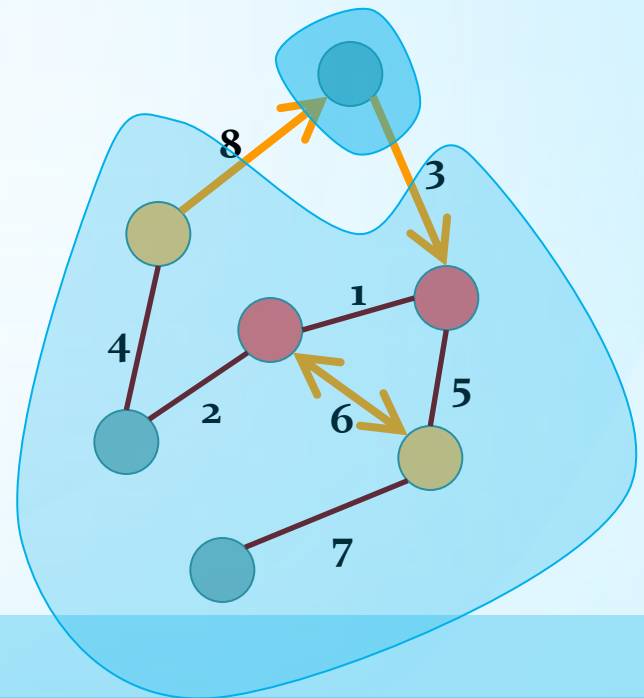
Branch – an MST edge



The Algorithm: Execution

- Minimum outgoing edge discovery
 - On receiving the *Initiate* message a node tries
 - to find a minimum outgoing edge.

*Sends a **Test** message on
Basic edges (minimal first)*

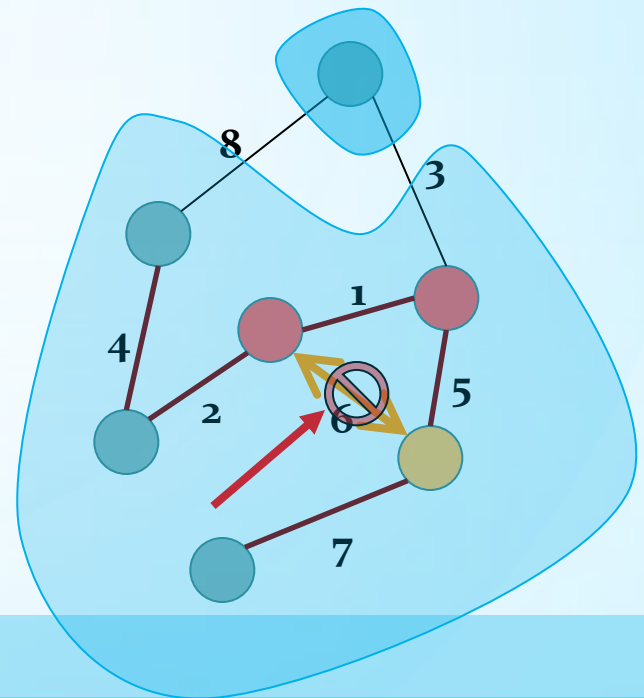


The Algorithm: Execution

- Minimum outgoing edge discovery
 - On receiving the **Test** message.

*In case of same identity:
send a **Reject** message, the
edge is **Rejected**.*

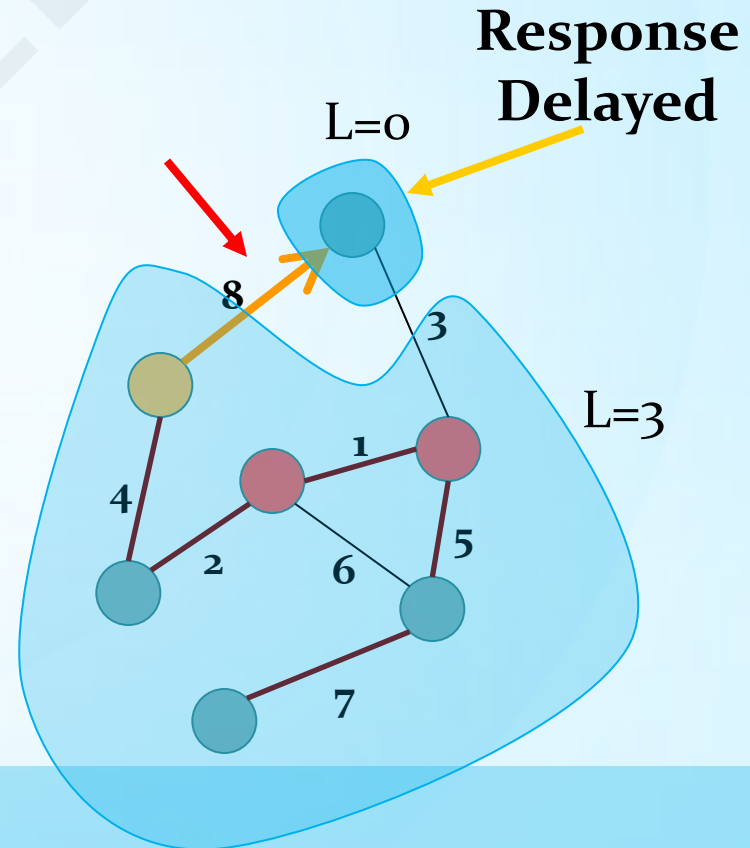
*In case **Test** was sent in
both directions, the edge is
Rejected automatically
without a **Reject** message.*



The Algorithm: Execution

- Minimum outgoing edge discovery
 - On receiving the **Test** message.

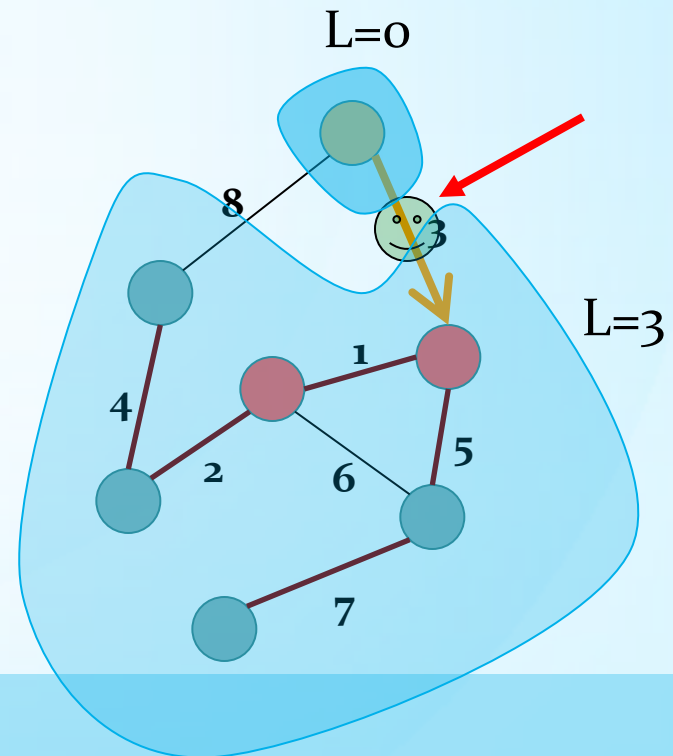
*In case of a self lower level:
Delay the response until the
identity rises sufficiently.*



The Algorithm: Execution

- Minimum outgoing edge discovery
 - On receiving the **Test** message.

*In case of a self higher level:
Send an **Accept** message
The edge is accepted as a
candidate.*

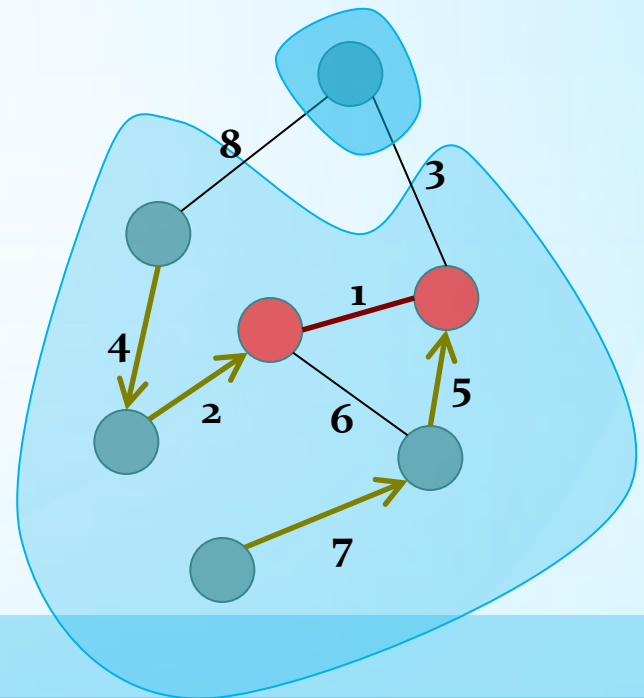


The Algorithm: Execution

- Minimum outgoing edge discovery
 - Agreeing on the minimal outgoing edge.

*Nodes send **Report** messages along the branches of the MST.*

If no outgoing edge was found the algorithm is complete.



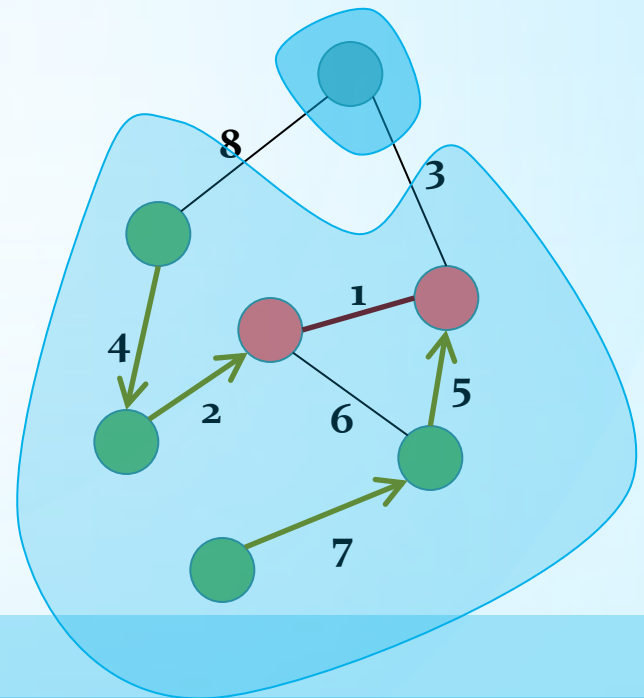
The Algorithm: Execution

- Minimum outgoing edge discovery
 - Agreeing on the minimal outgoing edge.

Nodes send **Report** messages along the branches of the MST.

If no outgoing edge was found the algorithm is complete.

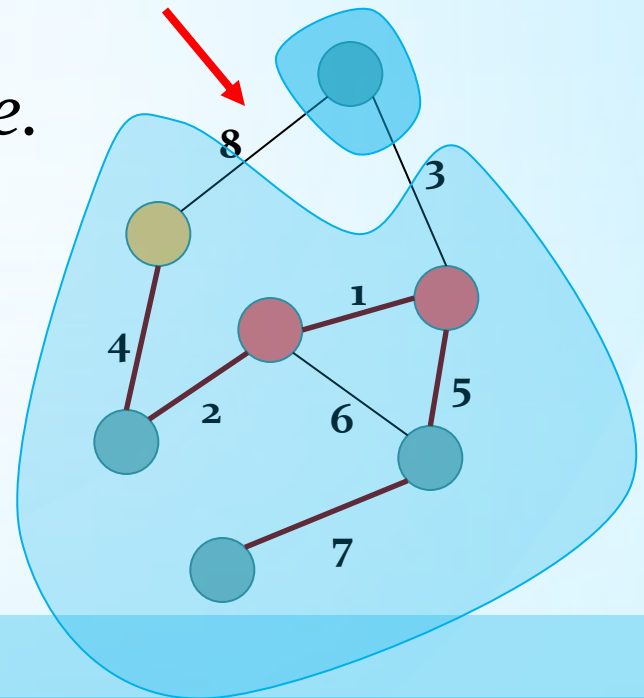
After sending they go into **Found** mode.



The Algorithm: Execution

- Minimum outgoing edge discovery
 - Agreeing on the minimal outgoing edge.

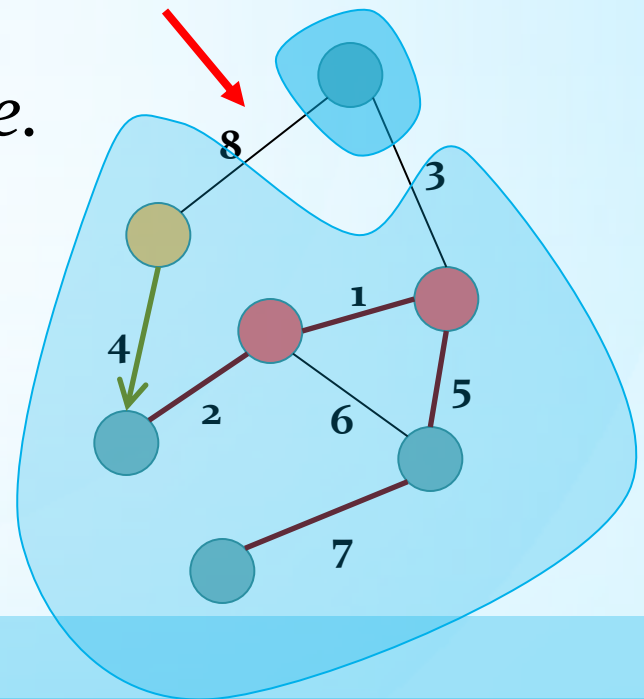
*Every leaf sends the **Report** when resolved its outgoing edge.*



The Algorithm: Execution

- Minimum outgoing edge discovery
 - Agreeing on the minimal outgoing edge.

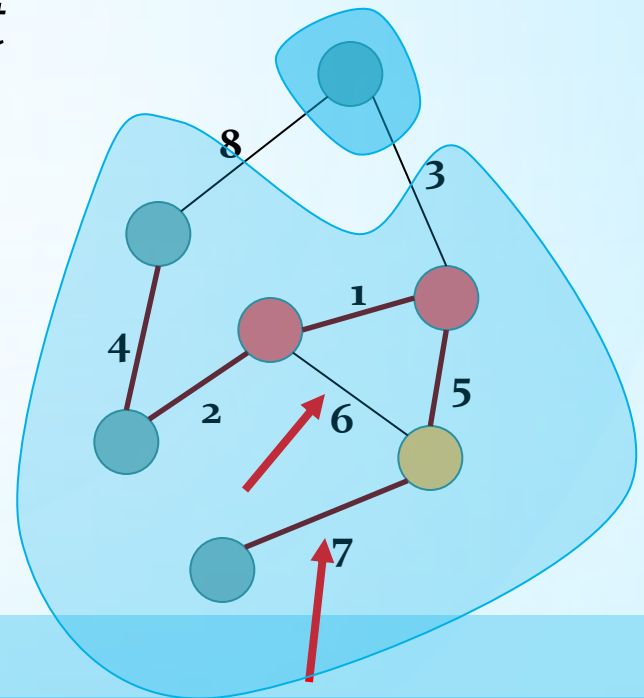
*Every leaf sends the **Report** when resolved its outgoing edge.*



The Algorithm: Execution

- Minimum outgoing edge discovery
 - Agreeing on the minimal outgoing edge.

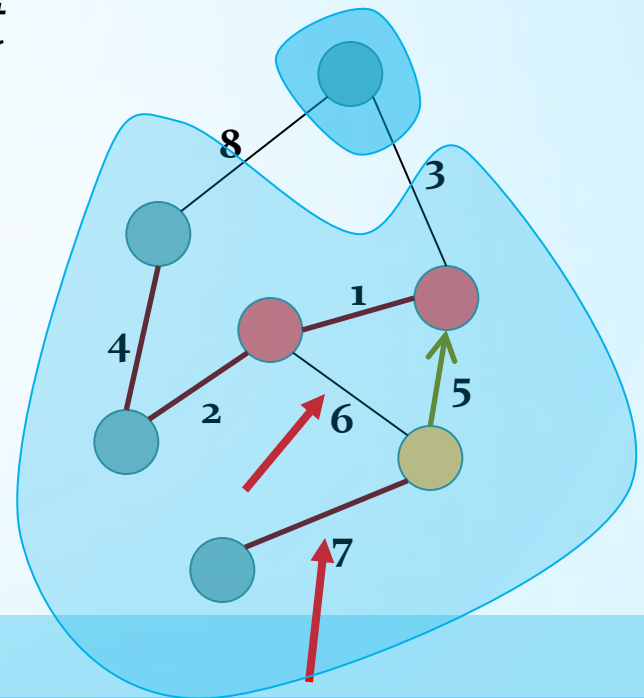
*Every interior sends the **Report** when resolved its outgoing and all its children sent theirs.*



The Algorithm: Execution

- Minimum outgoing edge discovery
 - Agreeing on the minimal outgoing edge.

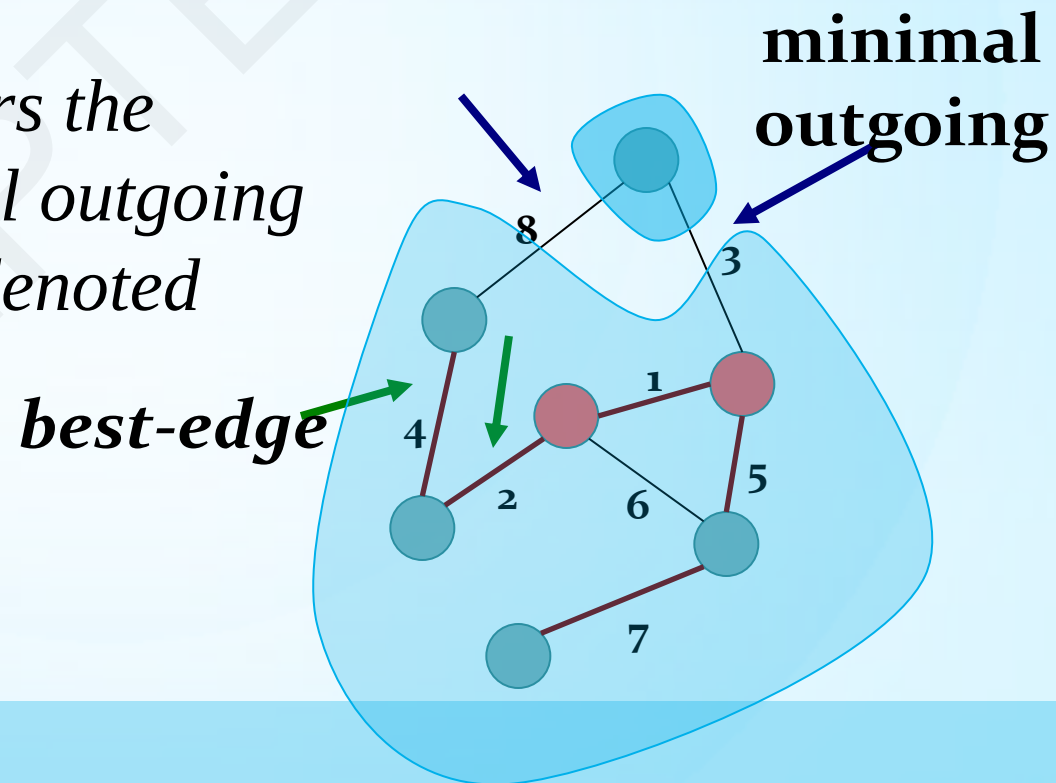
*Every interior sends the **Report** when resolved its outgoing and all its children sent theirs.*



The Algorithm: Execution

- Minimum outgoing edge discovery
 - Agreeing on the minimal outgoing edge.

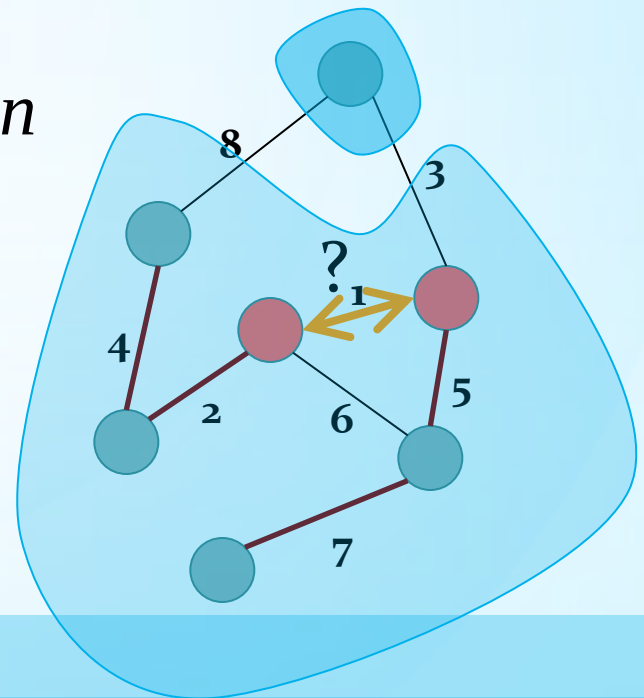
*Every node remembers the branch to the minimal outgoing edge of its sub-tree, denoted **best-edge**.*



The Algorithm: Execution

- Minimum outgoing edge discovery
 - Agreeing on the minimal outgoing edge.

*The core adjacent nodes exchange **Reports** and decide on the minimal outgoing edge.*

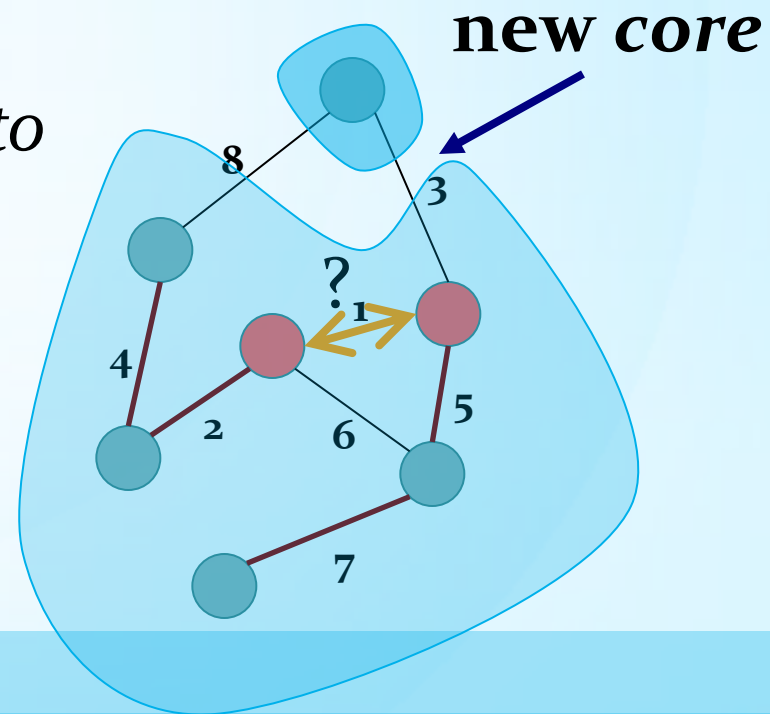


The Algorithm: Execution

- Combining segments
 - Changing core.

*When decided a **Change-core** message is sent over branches to the minimal outgoing edge.*

*The tree **branches** point to the new core.*



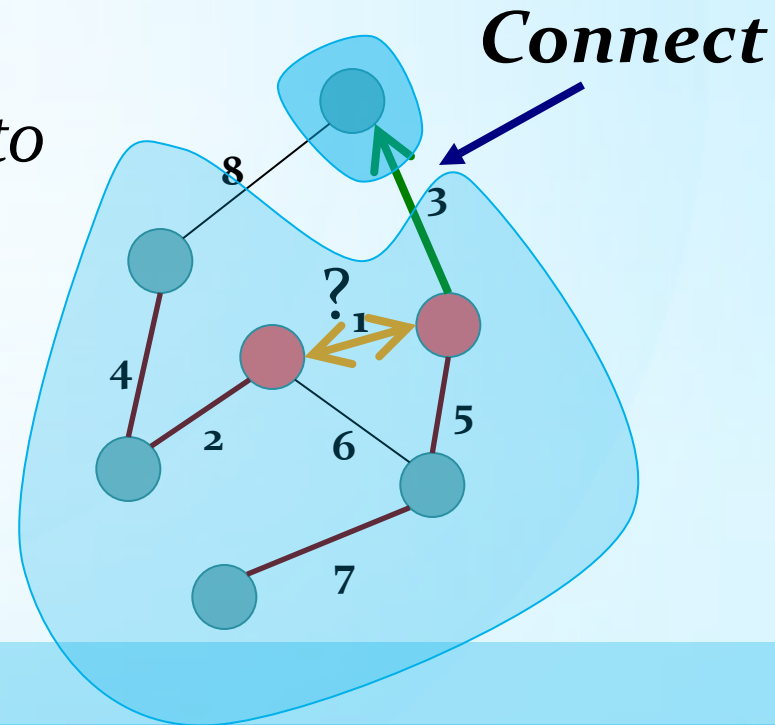
The Algorithm: Execution

- Combining segments
 - Changing core

*When decided a **Change-core** message is sent over branches to the minimal outgoing edge.*

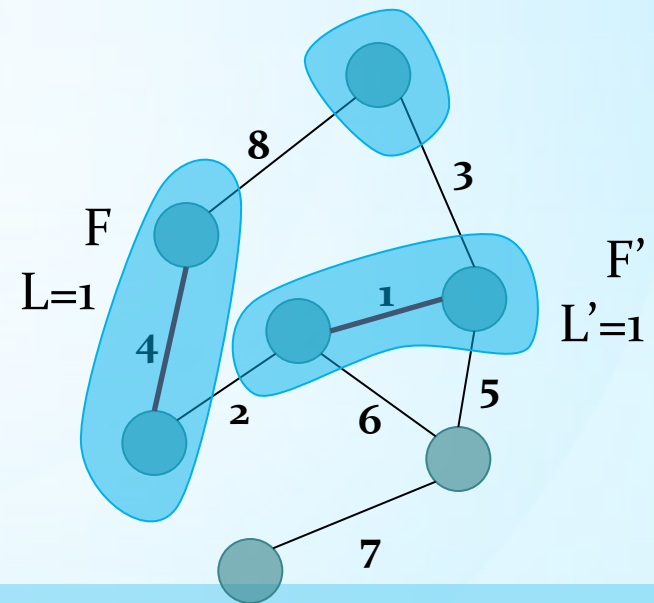
*The tree **branches** point to the new core.*

*Finally a **Connect** message is sent over the minimal edge.*



The Algorithm: Execution

- Final notes
 - Connecting same level fragments.

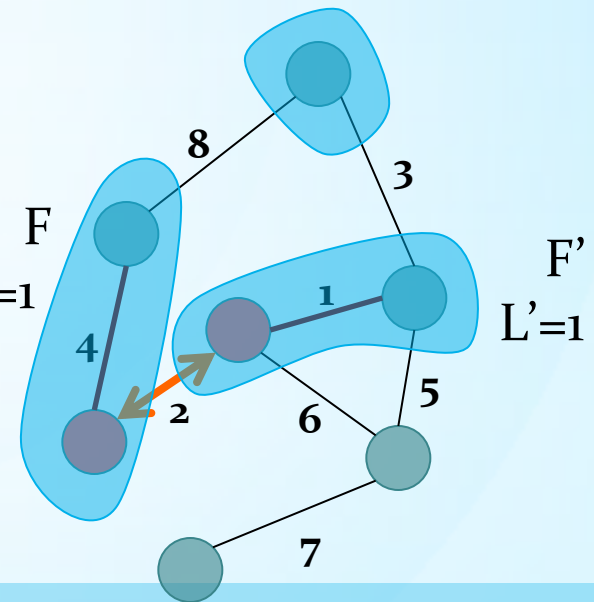


The Algorithm: Execution

- Final notes
 - Connecting same level fragments.

*Both core adjacent nodes send a **Connect** message, which causes the level to be increased.*

*As a result, core is changed and new **Initiate** messages are sent.*



The Algorithm: Execution

- Final notes
 - Connecting lower level fragments.

*When lower level fragment F' at node n' joins some fragment F at node n before n sent its **Report**.*

*We can send n' an **Initiate** message with the **Find** state so it joins the search.*

The Algorithm: Execution

- Final notes
 - Connecting lower level fragments.

*When lower level fragment F' at node n' joins some fragment F at node n after n sent its **Report**.*

*It means that n already found a lower edge and therefore we can send n' an **Initiate** message with the **Found** state so it doesn't join the search.*

The Algorithm: Execution

- Final notes
 - Forwarding the *Initiate* message at level L .

*When forwarding an **Initiate** message to the leafs, it is also forwarded to any pending fragments at level $L-1$, as they might be delayed with response.*

The Algorithm: Execution

- Final notes
 - Upper bound on fragment levels.

Level $L+1$ contains at least 2 fragments at level L .

Level L contains at least 2^L nodes.

$\log_2 N$ is an upper bound on fragment levels.

The Algorithm: Correctness

- Proof outline
 - Correct MST build-up

*The **Connect** message is sent on the minimal outgoing edge only.*

As a result of properties 1 & 2 we should obtain an MST.

The Algorithm: Correctness

- Proof outline
 - No deadlocks

Assume there is a deadlock.

Choose a fragment from the lowest level set and with minimal outgoing edge in that set.

*Its **Test/Connect** message surely will be replied.*

There will always be a working fragment.

The Algorithm: Complexity

- Complexity
 - Communication

*At most E **Reject** messages (with corresponding E **Test** messages, because each edge can be rejected only once.*

The Algorithm: Complexity

- Complexity
 - Communication

*At every but the zero or last levels, each node can accept up to 1 **Initiate**, **Accept** messages. It can transmit up to 1 **Test (successful)**, **Report**, **ChangeRoot**, **Connect**. Since the number of levels is bounded by $\log_2 N$ number of such messages is at most $5N(\log_2 N - 1)$.*

The Algorithm: Complexity

- Complexity
 - Communication

*At level zero, each node receives at most one **Initiate** and transmits at most one **Connect**. At the last level a node can send at most one **Report** message, as a result at most $3N$ such messages.*

The Algorithm: Complexity

- Complexity
 - Communication

As a result, the upper bound is: $5N \log_2 N + 2E$.

The Algorithm: Complexity

- Complexity

- Time (under assumption of initial awakening it is $5N \log_2 N$)

We prove by induction that one needs $5lN - 3N$ time units for every node to reach level l .

The Algorithm: Complexity

- Complexity

- Time (under assumption of initial awakening it is $5N \log_2 N$)

- $l=1 \rightarrow$ Each node is awakened and sends a **Connect** message. By time $2N$ all nodes should be at level 1.*

The Algorithm: Complexity

- Complexity

- Time (under assumption of initial awakening it is $5N \log_2 N$)

*Assume $l \rightarrow$ At level l , each node can send at most N **Test** messages which will be answered before time $5lN - N$. The propagation of the **Report** messages, **ChangeRoot**, **Connect**, and **Initiate** messages can take at most $3N$ units, so that by time $5(l + 1)N - 3N$ all nodes are at level $l + 1$.*

The Algorithm: Complexity

- Complexity
 - Time (under assumption of initial awakening it is $5N \log_2 N$)
*At the highest level only **Test**, **Reject**, and **Report** messages are used.*

The Algorithm: Complexity

- Complexity
 - Time (under assumption of initial awakening it is $5N \log_2 N$)

As a result we have the algorithm complete under $5N \log_2 N$ time units.

Conclusion

- **Distributed MST algorithms** are useful in communication networks when one wishes to broadcast information from one node to all other nodes and there is a cost associated with each channel of the network.
- In addition to the broadcast application, there are many potential control problems for networks whose communication complexities are reduced by having a known spanning tree.
- In this lecture, we have presented an **asynchronous distributed algorithm i.e. GHS algorithm** which determines the minimum-weight spanning tree (MST) of the graph.
- In upcoming lecture we will discuss about '**Termination Detection**' and set of representative termination detection algorithms.